D.K.T.E. SOCIETY'S TEXTILE AND ENGINEERING INSTITUTE, ICHALKARANJI

(An Empowered Autonomous Institute, Affiliated to Shivaji University, Kolhapur)

DEPARTMENT OF MCA

A PROJECT REPORT ENTITLED

ON

NAGARSETU AI: SMART CIVIC GRIEVANCE REDRESSAL & URBAN GOVERNANCE PLATFORM

SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE DEGREE OF MASTER OF COMPUTER APPLICATION

BY

Miss . Prerana Sunil Shinde

UNDER THE GUIDANCE OF

Prof. Shreyash Patil

DKTE
Promoting Excellence in
Teaching, Learning & Research

THROUGH THE DIRECTOR

D.K.T.E. S's TEXTILE AND ENGINEERING INSTITUTE,
ICHALKARANJI – 416115
ACADEMIC YEAR: 2026–2027

CERTIFICATE

This is to certify that the project report entitled "**NagarSetu AI: Smart Civic Grievance Redressal & Urban Governance Platform**" is a record of the project work carried out by **Miss. Prerana Sunil Shinde** in partial fulfillment of the requirements for the degree of Master of Computer Application, Semester III, at D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji.

The project work was carried out under the guidance of **Prof. Shreyash Patil** and is submitted in accordance with the academic requirements of the institute for the academic year **2026–2027**.

Project Guide	Head of the Department
Name: Prof. Shreyash Patil	**Name: _____**
Signature: _____	**Signature: _____**
Date: _____	**Date: _____**
Examiner 1: _____	**Examiner 2: _____**

Place: Ichalkaranji

GUIDE'S CERTIFICATE

I hereby certify that the project entitled "NagarSetu AI: Smart Civic Grievance Redressal & Urban Governance Platform" has been developed by **Miss. Prerana Sunil Shinde** under my guidance.

The student(s) have undertaken the project work, studied the relevant concepts of Generative Artificial Intelligence, implemented the proposed solution, and prepared the accompanying documentation.

The work has been reviewed by me and is submitted for academic evaluation. The project has been carried out under my supervision, subject to the institute's applicable academic integrity requirements.

Project Guide

Name: **Prof. Shreyash Patil**

Designation: _____

Signature: _____

Date: _____

STUDENT DECLARATION

To, The Registrar, Shivaji University, Kolhapur.

We, the undersigned, hereby declare that the project report entitled "**NagarSetu AI: Smart Civic Grievance Redressal & Urban Governance Platform**", submitted under the guidance of **Prof. Shreyash Patil** is our original work carried out as part of the Master of Computer Application programme at D.K.T.E. Society's Textile and Engineering Institute, Ichalkaranji.

We further declare that the empirical findings and results presented in this report are based on the work and information collected by us. We have not copied this report from another report submitted to Shivaji University, Kolhapur. All external sources, software libraries, datasets, APIs, and Generative AI tools used have been acknowledged. Any AI-generated content or code has been reviewed and verified by us, in accordance with applicable academic-integrity requirements.

We understand that plagiarism or misrepresentation may be dealt with under the applicable university and institute rules.

The project has been developed by us under the guidance of the assigned project guide. The information, results, screenshots, and observations presented in this report are accurate to the best of our knowledge.

All external sources, research papers, software libraries, datasets, APIs, and AI tools used in the project have been appropriately acknowledged. The use of Generative AI tools, including AI-generated code, content, images, or other outputs, has been disclosed wherever applicable.

We have not knowingly presented another person's work as our own. This project has not been submitted elsewhere for academic evaluation unless explicitly permitted by the institute.

Student Name(s): **Miss. Prerana Sunil Shinde**

Roll Number(s): **25PMC048**

Signature(s): _____

Date: _____

ACKNOWLEDGEMENT

We have been fortunate to receive assistance, suggestions, and support from our project guide, faculty members, friends, and the institute during the development of our project entitled "**NagarSetu AI: Smart Civic Grievance Redressal & Urban Governance Platform**".

First and foremost, we express our sincere gratitude to our respected project guide, **Prof. Shreyash Patil**, for valuable guidance, constructive suggestions, and kind support throughout the project work.

We take this opportunity to thank the Head of the Department, all teaching and non-teaching staff members, and our friends for their cooperation and encouragement.

We also sincerely thank the staff of the computer laboratory for their cooperation and for providing the facilities required to complete the project within the available time.

We acknowledge the developers and providers of software tools, Generative AI models, libraries, APIs, datasets, and technical resources used in the project, wherever applicable.

Miss. Prerana Sunil Shinde

ABSTRACT

This project presents NagarSetu AI, a full-stack smart civic grievance redressal and urban governance platform developed using Generative Artificial Intelligence for the Ichalkaranji Municipal Corporation (IMC). Existing municipal grievance mechanisms rely on paper petitions, telephonic helplines and generic web forms, which cause manual triage delays, misrouting between departments, duplicate work orders, closure of tickets without proof of work, and no automated tracking of service-level deadlines.

To address these problems, the proposed system uses AI components to classify citizen complaints, estimate severity, assess uploaded photographs, detect duplicate reports and recommend routing to the responsible department and prabhag (ward). A conversational assistant in English and Marathi guides citizens through filing and tracking complaints. Every complaint follows a closed-loop lifecycle of AI triage, controller-validated dispatch, field work with geotagged before-and-after evidence, supervisor review and citizen verification, and an append-only audit log records every action.

The application is developed using React 18, Vite, Tailwind CSS, Node.js, Express.js, MongoDB and Mongoose, with JWT-based role-based access control for seven user roles. The Generative AI component is used for intent classification, title generation, image assessment and the conversational assistant, while routing, SLA monitoring, duplicate-matching thresholds and access control are implemented with conventional programming.

The system provides a citizen portal, field worker terminal, officer desk, a controller command centre with a GIS map, SLA monitoring and analytics dashboards. It was evaluated through functional and API testing on a seeded dataset of more than 44 complaints and 29 work orders, and all thirteen test cases passed. The project aims to improve the speed, transparency and accountability of civic service delivery. It also identifies limitations, such as simulated SMS and WhatsApp delivery and the absence of a live deployment, and opportunities for future enhancement.

TABLE OF CONTENTS

Sr. No.	Contents	Page No.
1	Chapter 1: Introduction	1
2	Chapter 2: Existing System and Problem Analysis	3
3	Chapter 3: Proposed System	6
4	Chapter 4: Generative Artificial Intelligence Components Used	11
5	Chapter 5: System Analysis and Design	16
6	Chapter 6: Hardware and Software Requirements	26
7	Chapter 7: Implementation	27
8	Chapter 8: Testing and Results	33
9	Chapter 9: Scope, Limitations and Future Enhancements	36
10	Chapter 10: Conclusion	37
11	Bibliography / References	38
12	Appendices	39

LIST OF FIGURES

Figure No.	Figure Title	Page No.
Fig. 2.1	Workflow of the Existing Grievance Handling Process	3
Fig. 3.1	Seven-Step Closed-Loop Complaint Lifecycle	6
Fig. 3.2	Proposed System Architecture with the Generative AI Component	7
Fig. 5.1	System Architecture	16
Fig. 5.2	Use Case Diagram	17
Fig. 5.3	Data Flow Diagram: Level 0 (Context Diagram)	18
Fig. 5.4	Data Flow Diagram: Level 1	19
Fig. 5.5	Flowchart: Complaint Intake, AI Triage and Dispatch	20
Fig. 5.6	Flowchart: Field Execution, Verification and Closure	21
Fig. 5.7	Complaint Status State Diagram (principal transitions)	22
Fig. 5.8	Entity-Relationship Diagram	23
Fig. 7.1	Application Home Screen	29
Fig. 7.2	AI Output Screen (AI Triage Result)	30
Fig. 7.3	Controller Command Center with GIS Map	30
Fig. 7.4	Field Worker Evidence Upload	31
Fig. 7.5	Analytics Dashboard and Audit Log	31

LIST OF TABLES

Table No.	Table Title	Page No.
Table 2.1	Existing System Analysis	4
Table 2.2	Summary of the Literature Reviewed	4
Table 2.3	Comparison of the Existing and Proposed Systems	5
Table 3.1	Proposed System Features	7
Table 3.2	SLA Targets by Priority	8
Table 3.3	Module Description	9
Table 4.1	Generative AI Model and Service Details	11
Table 4.2	Generative AI Components	11
Table 4.3	Prompt Design Elements	12
Table 4.4	Model Parameters and System Thresholds	13
Table 4.5	Role of Generative AI and Conventional Programming	14
Table 5.1	Data Dictionary: Municipality Collection	23
Table 5.2	Data Dictionary: Complaint Collection	23
Table 5.3	Data Dictionary: WorkOrder Collection	24
Table 5.4	Data Dictionary: AuditLog Collection (Append-Only)	24
Table 5.5	Input and Output Design of the Main Screens	25
Table 5.6	Role-Based Access Control Matrix	25
Table 6.1	Hardware Requirements (Development and Testing)	26
Table 6.2	Software Requirements	26
Table 8.1	Test Cases and Results	33
Table 8.2	AI Output Evaluation Criteria	34
Table 8.3	Observed Performance Measures	35
Table A.1	Key API Endpoints	39
Table B.1	Prompt Library	39
Table B.2	Prompt Refinement Log	40
Table C.1	Sample Input and Output (Test Case TC-03)	40
Table C.2	Sample Civic Assistant Exchange	40
Table E.1	Environment Settings	41
Table G.1	Seeded Dataset	42

ABBREVIATIONS AND TERMINOLOGY

Abbreviation	Full form / meaning
AI	Artificial Intelligence
GenAI	Generative Artificial Intelligence
LLM	Large Language Model
NLP	Natural Language Processing
API	Application Programming Interface
REST	Representational State Transfer
JSON	JavaScript Object Notation
JWT	JSON Web Token
RBAC	Role-Based Access Control
ODM	Object Document Mapper (Mongoose)
UI / UX	User Interface / User Experience
ULB	Urban Local Body
IMC	Ichalkaranji Municipal Corporation
Prabhag	Administrative ward of a municipal corporation
SLA	Service Level Agreement
GIS	Geographic Information System
GPS	Global Positioning System
OTP	One-Time Password
KPI	Key Performance Indicator
ER	Entity-Relationship
DFD	Data Flow Diagram
PWA	Progressive Web App
CPGRAMS	Centralized Public Grievance Redress and Monitoring System
DARPG	Department of Administrative Reforms and Public Grievances
MoHUA	Ministry of Housing and Urban Affairs

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Project

Urban local bodies (ULBs) in India are responsible for roads, water supply, street lighting, drainage and solid waste management. Citizens use these services every day and, when something fails, they depend on the municipal grievance mechanism to have it corrected. In many Tier-2 and Tier-3 cities this mechanism still consists of paper petitions, telephonic helplines and basic web forms, with little automation between the moment a citizen reports a problem and the moment it is physically fixed.

NagarSetu AI ("Civic Bridge Powered by AI") is a full-stack smart civic operations and grievance redressal platform that addresses this gap. It is configured for the **Ichalkaranji Municipal Corporation (IMC)**, an industrial textile centre in Maharashtra, and turns grievance handling into a closed-loop, data-driven process. Citizens report issues through text, voice or photographs. AI components classify the issue, estimate its severity, detect duplicate reports and recommend a route. Work orders are dispatched to field crews, and a complaint is closed only after geotagged before-and-after evidence has been reviewed by a supervisor and confirmed by the citizen.

The platform serves four groups of users: citizens, field workers, ward and department officers, and the command-centre controllers and senior authorities who oversee the city. Every administrative action is stored in an append-only audit ledger, so the complete history of any complaint can be reviewed later.

1.2 Background of the Project

Digital grievance systems already exist in India. The Centralized Public Grievance Redress and Monitoring System (CPGRAMS), developed and monitored by the Department of Administrative Reforms and Public Grievances (DARPG), is an online platform available round the clock for lodging and tracking grievances about public service delivery [2]. The Swachhata-MoHUA application, the official app of the Ministry of Housing and Urban Affairs, lets citizens photograph a sanitation-related problem and forwards the geo-located complaint to the concerned municipal official [3]. These platforms show that citizens are ready to use digital channels. Their emphasis, however, is on lodging and tracking complaints rather than on verified, ward-level field execution inside a single municipal corporation.

Three developments make a more intelligent system feasible. First, transformer-based language models [7] can read free-form text and extract the intent of a message. Second, multimodal models can analyse photographs and judge whether they show real civic infrastructure damage. Third, open geospatial tools such as OpenStreetMap map tiles [19] and the Leaflet library [14], together with the GPS sensor and camera of an ordinary smartphone, allow every complaint to be placed on a map and clustered with nearby reports. NagarSetu AI combines these capabilities with a conventional web stack (React, Node.js, Express.js and MongoDB) to produce a working civic operations platform.

1.3 Motivation

The motivation for the project comes from the national vision of minimum government and maximum governance and from the goals of the Smart Cities Mission, launched in 2015 to promote sustainable, inclusive and citizen-centric urban development [1]. Citizens use highly responsive commercial platforms for travel, shopping and payments, yet reporting a pothole or a broken streetlight can still involve queues, phone calls and uncertain follow-up.

The project was selected because the problem is concrete, locally relevant to Ichalkaranji, and well suited to a combination of automation and Generative AI. Sorting and routing are repetitive clerical tasks, understanding free-text complaints is a language task, and verifying completion from photographs is a visual task. The project also offered the opportunity to build a complete multi-role application rather than a single AI feature.

1.4 Problem Statement

To design and develop a full-stack civic grievance redressal platform using Generative Artificial Intelligence to address manual triage delays, misrouting between departments, duplicate work orders, closure of tickets without proof of work, and untracked service-level deadlines, by providing AI-assisted multimodal complaint intake, automated routing, closed-loop evidence-based verification and an immutable audit trail to the citizens, field workers, officers and administrators of the Ichalkaranji Municipal Corporation.

1.5 Objectives of the Project

- To study existing municipal grievance redressal methods and identify their bottlenecks.
- To accelerate complaint intake through multimodal input (text, voice and photograph) with AI auto-fill and automatic location pinning.

- To automate classification, priority grading and routing of complaints using AI components, with human-in-the-loop validation.
- To prevent duplicate work orders by grouping nearby reports of the same incident under one parent incident.
- To enforce accountability through Service Level Agreement (SLA) countdowns, at-risk alerts and tiered escalation.
- To guarantee verified completion through geotagged before-and-after evidence, supervisor review and citizen sign-off.
- To give administrators live analytics on inflow, resolution velocity, ward load and SLA compliance, and to record every action in an append-only audit log.
- To evaluate the application through functional, API and workflow testing, and to identify its limitations and opportunities for improvement.

1.6 Scope of the Project

- **Intended users:** citizens, municipal field workers, ward officers, department officers, controllers of the command centre, senior authorities (Municipal Commissioner level) and system administrators.
- **Application domain:** municipal grievance redressal and urban operations for the Ichalkaranji Municipal Corporation, covering the 12 prabhags (wards) configured in the system and four civic domains: roads, water supply and drainage, electrical and street lighting, and solid waste management.
- **Main features:** AI-assisted complaint intake, triage, duplicate detection, routing and dispatch, SLA monitoring and escalation, evidence-based verification, a conversational assistant, GIS map and analytics dashboards, and an append-only audit log.
- **AI capabilities:** intent classification and title generation from text, image assessment, semantic duplicate detection and a bilingual (English and Marathi) conversational assistant.
- **Supported platforms:** a web application used from desktop browsers and from mobile browsers by field crews.
- **Boundaries:** the system is configured for a single municipal corporation and was evaluated on a seeded dataset. SMS and WhatsApp delivery run in demonstration mode, and offline synchronisation for field workers is simulated (see Section 9.2).

CHAPTER 2: EXISTING SYSTEM AND PROBLEM ANALYSIS

2.1 Overview of the Existing System

In traditional municipal set-ups, including those of many Tier-2 and Tier-3 municipal corporations, grievance handling relies on three main channels:

- **In-person physical petitions:** citizens travel to the corporation headquarters or a ward office and submit a paper application.
- **Telephonic helplines:** call centres record complaints in manual spreadsheets or legacy database forms with minimal validation.
- **Generic web forms and portals:** basic online forms accept unverified text, with no automatic geocoding of the location and no analysis of any attached photograph.

National platforms such as CPGRAMS [2] and the Swachhata-MoHUA app [3] add a digital channel for lodging and tracking complaints. Inside the municipal corporation, however, the steps that follow registration (triage, assignment, field work, verification and closure) are still largely manual.

2.2 Working of the Existing System

The typical process, shown in Fig. 2.1, runs as follows:

1. The citizen notices a civic problem and reports it by visiting the office, calling the helpline or submitting a web form.
2. A clerk or call-centre operator logs the complaint manually in a register, spreadsheet or legacy database form.
3. The complaint is reviewed manually to decide which department and ward are responsible.
4. The ticket is forwarded to the department, sometimes to the wrong one when responsibilities overlap.
5. An engineer assigns a field crew through a verbal or paper instruction.
6. The crew carries out the work. No photograph, GPS location or timestamp is required as evidence.
7. The crew informs the supervisor verbally that the work is complete.
8. The ticket is closed administratively, and the citizen is not asked to confirm that the problem has been fixed.

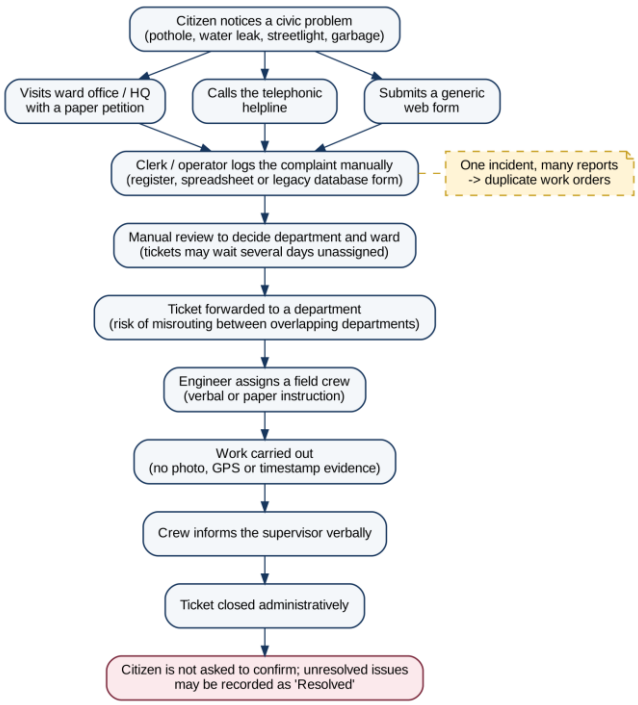


Fig. 2.1: Workflow of the Existing Grievance Handling Process

2.3 Limitations of the Existing System

Analysis of the existing process shows six main bottlenecks, summarised in Table 2.1.

Table 2.1: Existing System Analysis

#	Bottleneck	Existing system reality	Impact on municipal governance
1	**Manual triage delay**	Human operators review each ticket to find the relevant department and ward.	Tickets may linger unassigned for several days before reaching field crews.
2	**Misrouting errors**	Complaints are misclassified between overlapping departments, for example stormwater drainage and underground sewerage.	Tickets bounce between desks, and responsibility is disputed, which delays service.
3	**Duplicate flooding**	When a water main bursts or a road caves in, many citizens report the same incident.	Duplicate work orders waste resources and create conflicting assignments.
4	**No proof of work**	Field workers report completion verbally, and tickets are closed administratively.	Unresolved issues are marked "Resolved", which erodes public trust.
5	**No dynamic SLA tracking**	Deadlines are static entries in spreadsheets, with no countdown, warning or escalation trigger.	SLA breaches go unnoticed and nobody is held accountable.
6	**No spatial (GIS) visibility**	Complaints are isolated database rows without coordinates or clustering.	Authorities cannot see hotspots, failing infrastructure zones or crew dispatch patterns.

2.4 Literature Review

Table 2.2 reviews the policy documents, existing applications and research sources that informed the design of NagarSetu AI. The bracketed numbers refer to the Bibliography.

Table 2.2: Summary of the Literature Reviewed

Ref.	Author / organization, year	Title	Contribution	Relevance / limitation
[1]	Ministry of Urban Development, Government of India, 2015	Smart Cities Mission Statement and Guidelines	Sets out the objectives and strategy of the Smart Cities Mission, including citizen-centric, technology-enabled urban governance.	Provides the policy context for the project. It is a policy framework and does not specify a grievance-handling tool.
[2]	DARPG / Press Information Bureau, Government of India, 2026	Centralised Public Grievance Redress and Monitoring System (CPGRAMS)	Describes a 24×7 online platform, developed and monitored by DARPG, for lodging and tracking grievances and for monitoring redressal; reports the Next Generation CPGRAMS initiative with AI-based features.	Shows national acceptance of digital grievance channels. Its scope is grievances to public authorities in general, not field dispatch and verification inside one municipal corporation.
[3]	Ministry of Housing and Urban Affairs, Government of India	Swachhata-MoHUA mobile application	Lets citizens report civic issues with a geo-located photograph. The complaint goes to the concerned municipal official, and the citizen can follow its status and reopen it if unsatisfied.	A close precedent for photo-based reporting and reopening. Its scope is sanitation and cleanliness complaints under the Swachh Bharat Mission (Urban), and the sources reviewed do not describe AI triage or two-party verification.
[4]	T. Peixoto and M. L. Sifry (eds.), World Bank, 2017	Civic Tech in the Global South: Assessing Technology for the Public Good	Studies 23 platforms that amplify citizen voices and concludes that they appear to improve providers' capacity to respond but have yet to change their inclination to respond.	Supports the project's emphasis on accountability mechanisms (SLA timers, escalation, verification) rather than on reporting alone.
[5]	World Bank, 2022	CivicTech: Transparency, Engagement, and Collaboration for Better Governance	Reviews how digital technologies and data can involve citizens in the design and delivery of services, with case examples such as citizen apps and chatbots for service delivery monitoring.	Supports the use of a citizen app and chatbot. It is a general review and not a municipal grievance specification.

Ref.	Author / organization, year	Title	Contribution	Relevance / limitation
[6]	World Bank, 2024	Cross-Country Empirical Analysis of GovTech Platforms on Citizen Engagement (Policy Research Working Paper 10716)	Provides cross-country evidence that GovTech platforms foster citizen engagement, with stronger effects where government is more efficient and institutional fragility is low.	Indicates that platform success depends on institutional follow-through, which motivates human oversight and an audit trail.
[7]	A. Vaswani et al., 2017	Attention Is All You Need	Introduces the transformer architecture on which modern language models are built.	Technical foundation for the NLP and multimodal components. It does not address civic applications.

2.5 Comparative Analysis

Table 2.3: Comparison of the Existing and Proposed Systems

Feature	Existing system	Proposed system (NagarSetu AI)
User interaction	Paper petitions, helpline calls and generic web forms.	Web portal with text, voice and photo intake, AI auto-fill, and a conversational assistant in English and Marathi.
Automation	Manual triage, routing and assignment by clerks and engineers.	Automated classification, severity grading, duplicate grouping and routing recommendation, with SLA timers and automatic escalation.
AI integration	None in the typical municipal set-ups described.	Generative AI for intent classification, title generation, image assessment and the civic assistant, combined with rule-based routing.
Personalization	The same form and process for every user.	Role-specific portals for seven roles, personal complaint history, and a language toggle.
Response generation	Manual replies and static acknowledgements.	AI-generated titles, summaries and assistant replies, with status updates for each stage.
Data management	Isolated spreadsheet or database rows without coordinates.	MongoDB collections with coordinates, status history, SLA state and an append-only audit log, plus aggregation-based analytics.
Limitations	Delay, misrouting, duplicates, no proof of work, no SLA tracking, no GIS view.	Dependence on a third-party AI service and internet access, simulated SMS/WhatsApp gateway, and evaluation on seeded data only.

2.6 Problem Identification

The analysis identifies the following major problems:

- Complaints wait for manual triage and are sometimes sent to the wrong department.
- Several reports of one incident produce duplicate work orders and conflicting assignments.
- Completion is claimed without objective evidence, so tickets can be closed prematurely.
- Deadlines are not monitored automatically, so there is no early warning and no escalation.
- Authorities have no spatial view of complaint hotspots or ward-level performance.
- Citizens receive little feedback and have no structured way to confirm or dispute a closure.

2.7 Need for the Proposed System

Modern urban governance needs an operating system that automates clerical routing, verifies physical execution, provides spatial analytics and keeps an unalterable record of actions. A new solution is justified because each bottleneck in Table 2.1 can be addressed by a specific mechanism: AI triage for delay and misrouting, geo-semantic duplicate detection for duplicate work orders, geotagged before-and-after evidence with dual verification for premature closure, SLA timers with escalation for missed deadlines, and a GIS dashboard for visibility. The expected improvement is faster routing, fewer duplicated and prematurely closed tickets, and greater transparency for citizens and authorities.

CHAPTER 3: PROPOSED SYSTEM

3.1 Introduction

NagarSetu AI introduces a **closed-loop smart governance architecture**. Every grievance follows a verifiable seven-step lifecycle, from citizen reporting to citizen-verified closure, and no step can be skipped. The system combines AI-assisted triage with human validation, so the AI speeds up decisions while accountable officers keep the final say.

3.2 Overview of the Proposed System

A citizen files a complaint by describing the problem in text or voice, uploading a photograph, or chatting with the civic assistant. The backend sends the complaint to the AI triage service, which extracts the failure category, proposes a title, estimates severity and confidence, assesses the photograph, and checks whether a similar complaint already exists within 100 metres. The routing logic then maps the category to a department, the GPS coordinates to a prabhag, and balances the load across field crews. A controller validates the recommendation, and a work order with an SLA timer is dispatched.

The field worker accepts the task, performs the work and uploads before-and-after photographs with GPS and notes. The supervisor reviews the evidence, and the citizen gives the final confirmation. If the citizen is not satisfied, the complaint is reopened and an administrative alert is raised. Generative AI contributes at the front of this process (understanding the complaint, assessing the image and assisting the citizen), while conventional programming handles dispatch, deadlines, access control and audit logging.

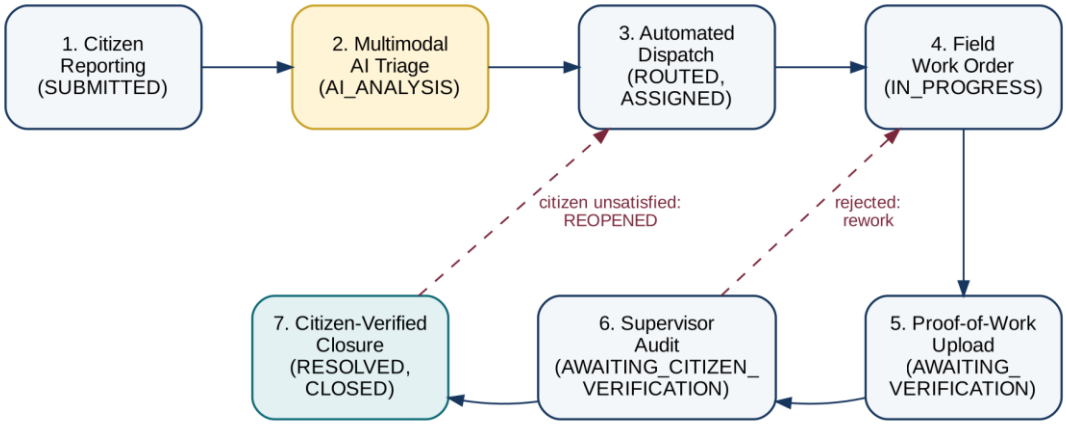


Fig. 3.1: Seven-Step Closed-Loop Complaint Lifecycle

3.3 Proposed System Architecture

Fig. 3.2 shows the flow of a request through the system: User → User Interface → Application Backend → AI Triage Service → Generative AI model/API → Response Processing → Controller validation → Output to the user, with MongoDB storing the results. A more detailed layered view is given in Section 5.1.

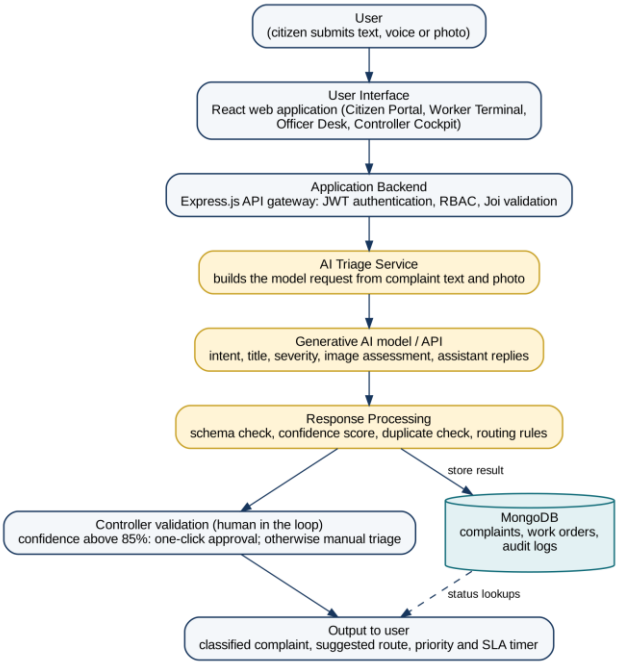


Fig. 3.2: Proposed System Architecture with the Generative AI Component

3.4 Major Features

Table 3.1: Proposed System Features

Feature	Description	Main users
Multimodal AI-assisted intake	Complaint through text, voice or photograph, with auto-suggested title and category and GPS-based identification of the prabhag.	Citizen
AI triage and classification	Intent extraction, severity grading (P1 to P4) and a confidence score for each complaint.	System, controller
Duplicate detection	Reports within 100 m with semantic similarity of at least 0.82 are linked to a parent incident without a new work order.	System
Routing and dispatch	Department and prabhag resolution, crew workload balancing and one-click controller validation.	Controller
Command centre	Live GIS map with ward-level incident density, telemetry cards (inflow, active complaints, P1 emergencies, SLA at risk) and the AI triage queue.	Controller, senior authority
SLA monitoring and escalation	Countdown timers per priority, at-risk alerts and automatic multi-level escalation.	Controller, officers
Field worker terminal	Task list, navigation to the site, start-work timestamp and camera upload of before-and-after evidence.	Field worker
Dual verification	GPS check within 50 m, supervisor review and citizen sign-off before closure; automatic reopening if the citizen is unsatisfied.	Worker, officer, citizen
Civic assistant	Chatbot that guides complaint filing, looks up status by complaint ID and answers civic questions in English and Marathi.	Citizen
Analytics	Weekly inflow, resolution velocity, category distribution, ward load and a city health score.	Officers, senior authority
Audit log	Append-only record of every action with actor, role, time and metadata.	Controller, administrator
Role-based access and theming	Seven roles with scoped data access; light and dark themes; command palette (Ctrl + K) for quick navigation.	All users

Table 3.2: SLA Targets by Priority

Priority	Severity	Target resolution time
P1	CRITICAL	4 hours
P2	HIGH	24 hours
P3	MEDIUM	48 hours
P4	LOW	72 hours

3.5 Functional Requirements

1. The system shall allow a registered citizen to submit a complaint with a description, an optional photograph or voice note, and a location.
2. The system shall validate required fields and reject invalid input with a clear message.
3. The system shall process each complaint through the AI triage service and store the category, priority, severity and confidence score.
4. The system shall detect probable duplicates within 100 m and link them to a parent complaint without creating a duplicate work order.
5. The system shall recommend the responsible department and prabhag and require controller validation when AI confidence is not above 85%.
6. The system shall create work orders with unique identifiers and start an SLA timer according to the priority.
7. The system shall raise at-risk alerts and escalate complaints that breach their SLA.
8. The system shall let field workers accept tasks, record the start time and upload before-and-after photographs with GPS data.
9. The system shall reject completion evidence whose GPS position is not within 50 m of the reported location.
10. The system shall require supervisor approval and citizen confirmation before a complaint is closed, and shall reopen the complaint if the citizen is not satisfied.
11. The system shall provide a conversational assistant for filing complaints and checking status.
12. The system shall restrict each user to the data and actions permitted by their role.
13. The system shall record every significant action in an append-only audit log.
14. The system shall display analytics and a GIS map of complaints to authorised staff.
15. The system shall handle API errors, authentication failures and invalid requests with structured error responses.

3.6 Non-Functional Requirements

- **Usability:** responsive layout for desktop and mobile, light and dark themes, English and Marathi support in the assistant, and keyboard navigation through the command palette.
- **Reliability:** centralised error handling in the backend and graceful messages in the interface; AI recommendations are always subject to validation, so a complaint does not depend on an unchecked AI decision.
- **Performance:** AI classification should complete within about one second so that triage does not delay intake; analytics use database aggregation pipelines.
- **Security:** JWT bearer authentication, hashed passwords, role-based data scoping, input validation and server-side storage of secrets.
- **Maintainability:** layered architecture with separate configuration, models, controllers, services and middleware.
- **Scalability:** every record carries a municipality identifier so that further municipal corporations can be added with data isolation; MongoDB can run as a replica set.
- **Accessibility:** high-contrast light theme and readable typography.
- **Privacy:** citizens see only their own complaints, officers see their ward or department, and personal data is not exposed to unrelated roles.

3.7 Module Description

Table 3.3: Module Description

Module	Description
User Interface	React single-page application with citizen portal, worker terminal, officer desk and controller cockpit; accepts input and displays results, charts and the GIS map.
Authentication and RBAC	Login, JWT issue and verification, and role-based data scoping for the seven roles.
Prompt Management	Builds the instructions sent to the AI model, including allowed categories, severity scale and required output format.
AI Integration (Triage Service)	Sends requests to the AI model or API for intent classification, title generation and image assessment, and receives the structured result.
Duplicate Detection	Combines Haversine distance and cosine similarity of text embeddings to link duplicate reports.
Routing and Dispatch	Maps category to department, coordinates to prabhag, balances crew workload and applies the confidence threshold.
Work Order Management	Creates work orders and manages acceptance, progress, evidence upload, supervisor review and rework.
SLA Engine	Computes breach times, updates SLA state (on track, at risk, breached) and triggers escalation.
Verification and Closure	Checks GPS and evidence, supervisor approval and citizen sign-off; reopens complaints when required.
Civic Assistant	Conversational interface for guided filing, status lookup and civic information.
Analytics	Aggregation pipelines for inflow, resolution velocity, category distribution, ward load and city health score.
Audit Logger	Writes append-only audit records for administrative and status-change events.
Database Module	Mongoose models and MongoDB collections for municipalities, wards, departments, users, complaints, work orders and audit logs.

3.8 Advantages of the Proposed System

- Triage and routing are completed by the system within about a second instead of waiting for manual review.
- Misrouting is reduced because the department and prabhag are resolved from the category and the GPS coordinates.
- Duplicate work orders are avoided by grouping nearby reports, which also raises the priority of widely reported incidents.
- Completion is backed by geotagged photographic evidence and two-party confirmation.
- SLA timers and automatic escalation make delays visible and assign accountability.
- A GIS map and analytics give decision-makers an immediate view of hotspots and ward performance.
- The append-only audit log supports transparency and later review.

3.9 Feasibility Study

Technical feasibility. The system uses widely available open-source technologies (React, Vite, Node.js, Express.js, MongoDB, Leaflet) and a cloud AI service accessed through an API. All of these were available to the team, and the required JavaScript, database and web development skills were covered in the MCA curriculum.

Economic feasibility. Development tools and libraries are free. The recurring costs are hosting (a 4 vCPU, 16 GB compute server and a managed MongoDB cluster are suggested in Chapter 6), the usage charges of the AI service, and a paid SMS/WhatsApp gateway if live messaging is enabled.

Operational feasibility. The roles in the system mirror the existing municipal hierarchy (citizen, field worker, ward officer, department officer, controller and commissioner), so staff can adopt it without changing their organisational responsibilities. Human validation of AI recommendations keeps authority with officers.

Schedule feasibility. The modular architecture allowed the backend, frontend and AI components to be developed in parallel, and the project was completed within the academic semester.

CHAPTER 4: GENERATIVE ARTIFICIAL INTELLIGENCE COMPONENTS USED

This chapter explains exactly where and how Generative AI is used in NagarSetu AI, and which parts of the system are implemented with conventional programming.

4.1 Introduction to Generative AI

Generative Artificial Intelligence refers to models that produce new content, such as text, images, audio or code, in response to an input prompt, instead of only assigning a label to the input. Large language models (LLMs) are trained on large text collections using the transformer architecture [7], and multimodal models extend this ability to images.

This capability is relevant to a civic grievance platform because complaints arrive as unstructured narrative text, sometimes in a regional language, and often with photographs. A generative model can read such input, extract structured meaning from it, write a concise title and answer questions in natural language. These tasks would otherwise need manual clerical effort or rigid form fields.

4.2 Generative AI Model Used

NagarSetu AI accesses a multimodal large language model through a provider API. The model is called only from the backend AI Triage Service and never from the browser, so access credentials stay on the server (Section 4.8). Visual defect assessment currently combines pre-trained vision heuristics with multimodal LLM APIs (Section 9.2).

Table 4.1: Generative AI Model and Service Details

Item	Details
Provider	Google Gemini
Model name / identifier	gemini-2.5-flash
Version	Gemini 2.5 Flash (accessed through the provider API)
Purpose	Intent classification, title generation, severity estimation, image assessment and civic assistant replies.
Access method	REST API called from the Node.js backend (AI Triage Service); credentials held in server environment variables.
Embedding model (duplicate detection)	Semantic text embeddings generated through the configured AI service; vectors are compared by cosine similarity in the backend
Speech-to-text (voice reports)	Browser speech recognition converts voice input to text before analysis

4.3 Generative AI Components

NagarSetu AI integrates five specialised AI components. Table 4.2 relates them to the standard component categories of a Generative AI application.

Table 4.2: Generative AI Components

Component	Description	Use in the project
Large Language Model	Generates or transforms text	Reads complaint text, extracts the category, writes titles and summaries, estimates severity and powers the civic assistant.
Multimodal (vision) model	Interprets images	Checks that a photograph shows civic infrastructure and identifies the damage pattern.
Prompt engineering	Structures instructions for the model	Prompts define the role, allowed categories, severity scale and structured output (Section 4.4, Appendix B).
API integration	Connects the application to the AI service	The AI Triage Service in the Node.js backend calls the provider API and processes the response.
Embeddings	Represent content as vectors	Used: complaint descriptions are converted to semantic vectors and compared by cosine similarity to find duplicates.

Component	Description	Use in the project
Vector database	Stores and retrieves vectors	No dedicated vector database is used; candidate complaints are first selected by distance and then compared.
Speech processing	Speech-to-text or text-to-speech	Used: voice reports are transcribed to text before analysis.
RAG	Grounds responses in retrieved information	Not implemented. Listed as a future enhancement.
AI agents	Select or execute tasks using tools	Not implemented.
Image generation	Generates images from prompts	Not implemented.

4.3.1 Component 1: Multimodal NLP and Semantic Intent Classification

This component processes raw citizen text and transcribed voice reports in English and regional vernacular. It uses transformer-based natural language processing with zero-shot classification, which means that categories can be assigned without training a separate classifier for each one. It extracts the core grievance intent (for example a pothole cavity, contaminated water leak or broken streetlight), generates an executive title from an informal narrative, computes a semantic confidence score (for example 94.2%) and assigns an initial severity of CRITICAL (P1), HIGH (P2), MEDIUM (P3) or LOW (P4).

4.3.2 Component 2: Computer Vision Defect and Damage Assessment

This component analyses photographs uploaded by citizens and field workers. It validates that the photograph depicts actual civic infrastructure, filtering out spam, selfies and unrelated images. It identifies physical damage patterns such as asphalt cracks, exposed wiring, water accumulation or unsegregated garbage. It also compares the "before" photograph with the worker's "after" photograph, using perceptual image hashing and feature similarity, to check that genuine restoration took place at the same location.

4.3.3 Component 3: Geo-Spatial Duplicate Detection and Clustering

This component prevents duplicate work orders. For an incoming complaint it finds existing complaints within a radius of 100 metres using the Haversine great-circle distance [9], $d = 2R \cdot \arcsin(\sqrt{(\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2)})$, where R is the Earth's radius and φ and λ are latitude and longitude. It then compares the semantic vector embeddings of the descriptions using cosine similarity. If the distance is at most 100 m and the similarity is at least 0.82, the new complaint is classified as a duplicate and linked as a child report to the primary parent incident. The parent's subscriber count increases, which raises its priority without creating another field work order.

4.3.4 Component 4: Intelligent Dispatch and Routing Matrix

This component routes a validated complaint to the right department, prabhag, supervisor and field crew. Department resolution maps the category to a department (road to Road Maintenance, water to Water Supply and Sewerage, electrical to Electrical and Lighting, waste to Solid Waste Management). Geographic resolution maps the GPS coordinates to one of the configured prabhags using polygon containment. Workload balancing evaluates the active work-order queue of each crew to avoid overloading a worker. For human-in-the-loop control, an AI confidence above 85% produces a recommended route that the controller can approve with one click, while a lower confidence flags the ticket for manual supervisor triage.

4.3.5 Component 5: 24/7 Conversational Civic Assistant

The NagarSetu Assistant is a chatbot available on every page. It guides citizens step by step through filing a grievance without complex dropdown selections, looks up the status of a complaint from its ID (for example NGS-2026-000143), and answers civic questions such as office hours, waste collection schedules, property tax guidelines and emergency contacts. It supports an English and Marathi language toggle.

4.4 Prompt Engineering and Design

Prompts for the AI Triage Service are designed so that the model returns a structured, validated result and does not guess when information is missing. Table 4.3 lists the design elements considered.

Table 4.3: Prompt Design Elements

Element	Purpose	Application in NagarSetu AI
Role / task definition	Tells the model what it is and what to do.	A civic complaint triage assistant for a municipal corporation.
Context	Supplies the domain knowledge needed.	Allowed categories, departments, severity scale P1 to P4 and the meaning of each level.

Element	Purpose	Application in NagarSetu AI
Input instructions	Explains how to read the input.	Complaint text in English or Marathi, optional photograph, location text.
Output format	Makes the answer machine-readable.	A fixed set of fields (category, title, severity, confidence, summary) returned as JSON.
Constraints	Limits unwanted behaviour.	Choose only from allowed values; do not invent details; state when the image or text is not about civic infrastructure.
Error handling	Defines behaviour on unclear input.	Low confidence or unclear input results in manual triage rather than automatic routing.

The difference between a basic and an improved prompt can be illustrated as follows. A basic prompt such as "Classify this complaint." leaves the categories, output format and handling of unclear cases to the model. An improved prompt, written in the style used in this project, is shown below.

```
Act as a civic complaint triage assistant for the Ichalkaranji Municipal
Corporation. Read the citizen's complaint (English or Marathi) and, if
provided, the attached photograph. Choose exactly one category from the
allowed list. Suggest a title of at most ten words, a severity (CRITICAL,
HIGH, MEDIUM or LOW) and a confidence between 0 and 1. If the text or image
does not describe civic infrastructure, say so. Respond only with JSON using
the keys: category, title, severity, confidence, summary.
```

The prompt above shows the structure used for triage. The prompt template implemented in the backend is given in Section 7.5, and the prompts of the application, with their purpose, inputs and refinements, are documented in Appendix B.

4.5 Workflow of the Generative AI Component

1. The citizen submits a complaint (text, voice or photograph) with a location.
2. The application validates the input (required fields, file type, location).
3. The AI Triage Service constructs the prompt from the complaint and the configured categories.
4. The request is sent to the AI model or service.
5. The model returns a structured analysis (category, title, severity, confidence, image assessment).
6. The application checks the response, runs duplicate detection and applies the routing rules and the 85% confidence threshold.
7. The controller reviews the recommendation in the AI triage queue and approves or overrides it.
8. The result, status history and audit record are stored, and the citizen sees the outcome and status updates.

4.6 Model Parameters and Configuration

Table 4.4 lists the model-level parameters and the system-level thresholds that control the AI behaviour.

Table 4.4: Model Parameters and System Thresholds

Parameter	Value	Note
Model identifier	gemini-2.5-flash	See Table 4.1
Temperature	Provider default	Model setting
Maximum output tokens	Provider default	Model setting
System instructions	See Appendix B	Prompt library
AI confidence threshold for recommended routing	Above 85%	System design; otherwise manual triage
Duplicate search radius	100 m	Haversine distance
Semantic similarity threshold	0.82 or higher	Cosine similarity of embeddings
GPS tolerance for completion evidence	50 m	Distance from reported coordinates

4.7 AI Output Validation

- **Confidence gating:** a recommendation is accepted for one-click approval only when the confidence is above 85%. Anything lower is flagged for manual supervisor triage.
- **Human in the loop:** every AI routing recommendation appears in the AI triage queue, where the controller can verify or override it, so a wrong classification does not dispatch a crew unchecked.
- **Schema enforcement:** category, priority, severity and status values must belong to the enumerations defined in the Mongoose schemas, so values outside the allowed sets are rejected.
- **Image checks:** the vision component filters spam, selfies and unrelated images, and the before-and-after comparison checks that restoration was genuinely carried out at the same location.
- **Duplicate safeguards:** a report is linked as a duplicate only when both the distance and the similarity conditions are met, which limits false matches.
- **Hallucination control in the assistant:** complaint status answers come from database records addressed by complaint ID, not from the model's memory.
- **API failures:** complaints can still be triaged manually if AI analysis is unavailable.

4.8 Responsible AI Considerations

- **Data privacy:** complaints may contain photographs, locations and contact details. Data is scoped by role, and only the information needed for a task is sent to the AI service.
- **Bias and fairness:** language models may perform unevenly on regional vernacular and on photographs from different localities. Marathi input and local conditions are therefore checked during testing, and low-confidence results are routed to humans.
- **Copyright and attribution:** the AI services, libraries and map data (OpenStreetMap contributors) used in the project are acknowledged in the references.
- **Human oversight:** the AI recommends and officials decide. Escalation, verification and closure remain with accountable people, and every action is audit-logged.
- **Misinformation:** civic information given by the assistant (office hours, schedules, tax guidelines) must be kept consistent with official municipal sources.
- **API key protection:** keys are stored in server environment variables, never in the frontend code or in the repository.
- **Ethical use:** the system is intended to improve service delivery and transparency, not to monitor individuals.

4.9 Role of Generative AI in the Project

Table 4.5 separates the tasks that genuinely depend on Generative AI or other machine-learning models from those implemented with conventional programming.

Table 4.5: Role of Generative AI and Conventional Programming

Task	Approach	Depends on GenAI?
Intent classification and title generation	LLM with zero-shot classification	Yes
Initial severity estimation (P1 to P4)	LLM output mapped to priority levels	Yes
Image validity and damage assessment	Multimodal model with vision heuristics	Yes
Text embeddings for duplicate detection	Embedding model	Yes (model); matching is arithmetic
Civic assistant (filing guidance, FAQs)	Conversational LLM	Yes
Voice transcription	Speech recognition	AI model, not generative
Before-and-after similarity check	Perceptual hashing and feature similarity	No (conventional computer vision)
Duplicate decision	Haversine distance and cosine similarity thresholds	No
Department, prabhag and crew routing	Rule mapping, polygon containment, workload balancing	No
SLA timers and escalation	Scheduled engine and rules	No
Authentication, RBAC, audit log, analytics	JWT, middleware, database aggregation	No

CHAPTER 5: SYSTEM ANALYSIS AND DESIGN

5.1 System Architecture Diagram

The system follows a three-layer architecture (Fig. 5.1). The presentation layer contains four role-based interfaces built with React. They communicate with the application backend through RESTful HTTP calls made with Axios. The backend is an Express.js API gateway that applies authentication, role-based scoping and error handling before routing requests to the complaint, work-order and analytics routers. These routers call the services for GenAI triage, SLA management and audit logging, which read and write MongoDB through Mongoose.

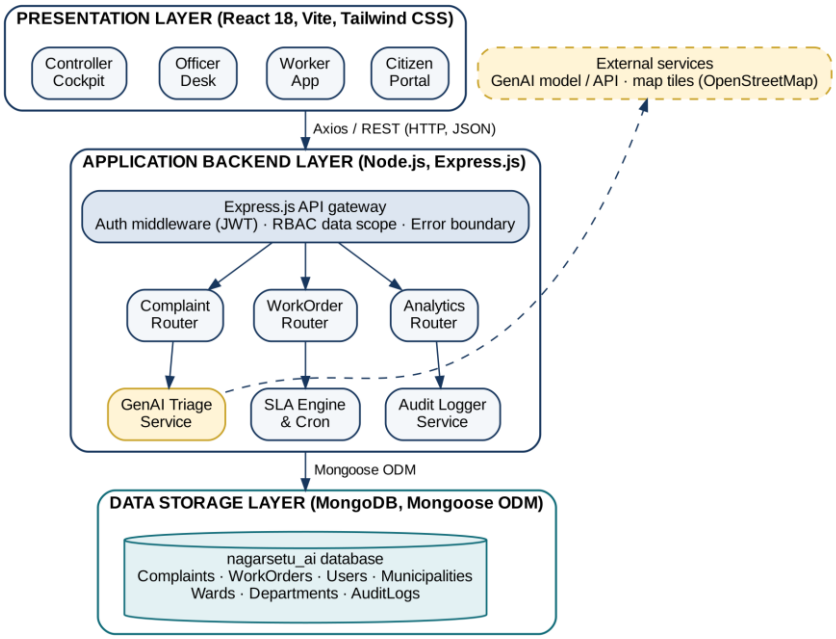


Fig. 5.1: System Architecture

5.2 Use Case Diagram

Fig. 5.2 shows the seven actors and the functions each can perform. Filing a complaint includes the AI triage and duplicate detection step. The controller validates AI recommendations, dispatches work orders and monitors SLAs, while senior authorities and administrators work at city-wide and platform level.

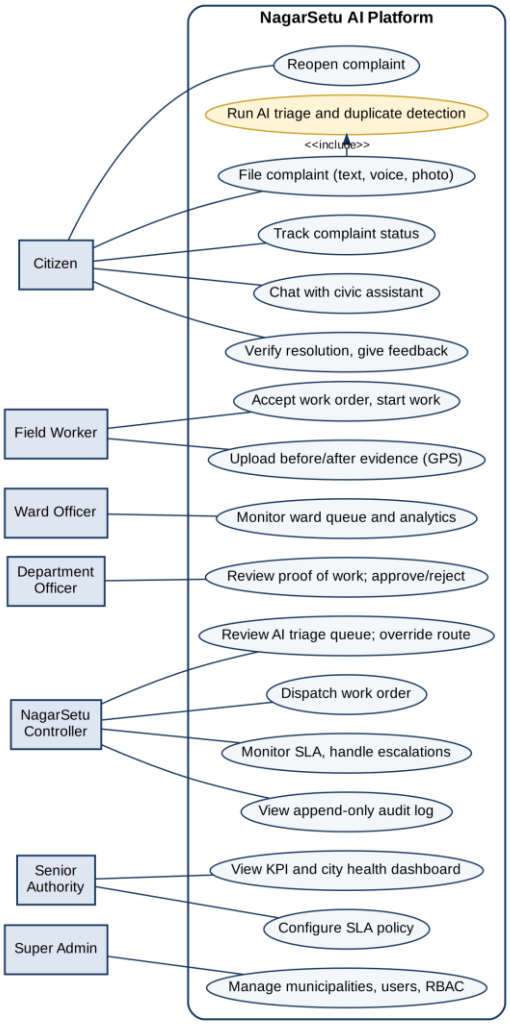


Fig. 5.2: Use Case Diagram

5.3 Data Flow Diagram (DFD)

Fig. 5.3 is the Level 0 (context) diagram. It shows NagarSetu AI as a single process exchanging data with citizens, field workers, officers, controllers, senior authorities, the GenAI model and the map service.

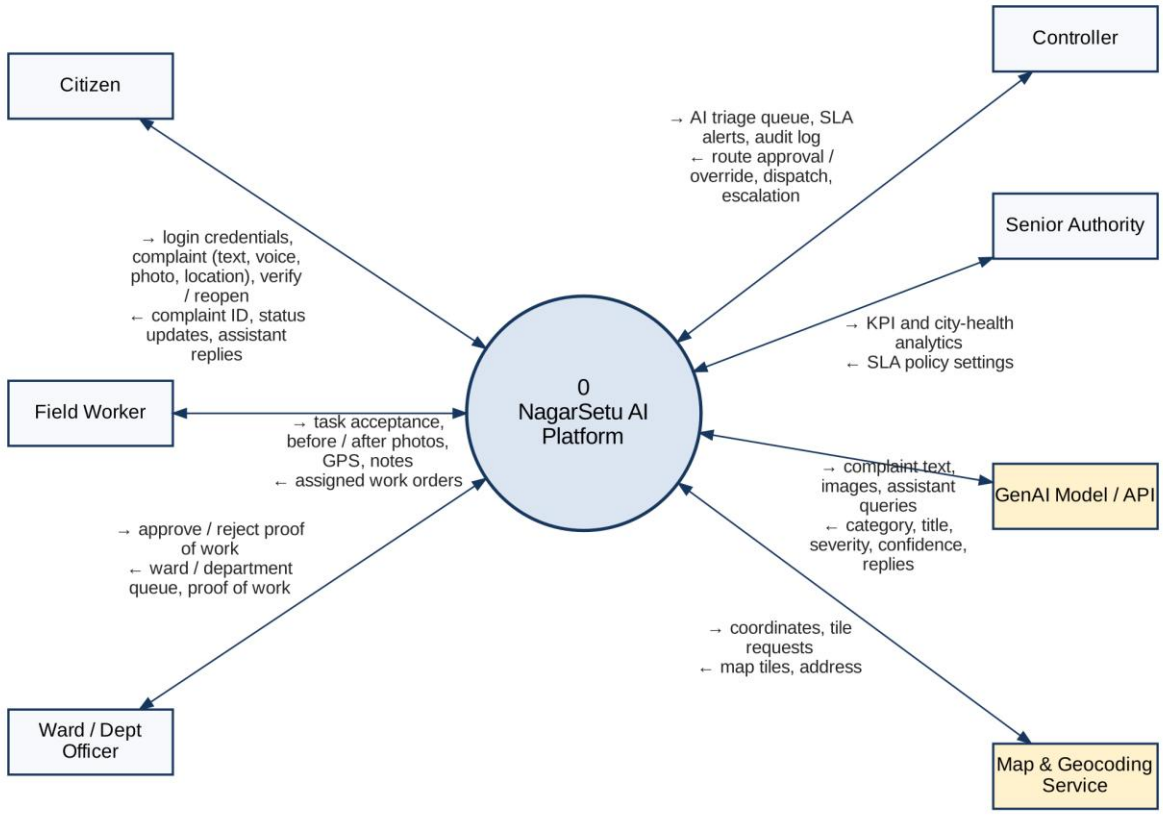


Fig. 5.3: Data Flow Diagram: Level 0 (Context Diagram)

Fig. 5.4 expands the system into six processes and three data stores. For readability, the audit events that processes 1.0 to 5.0 write to the AuditLogs collection, and the ward and department master data read by process 3.0, are not drawn.

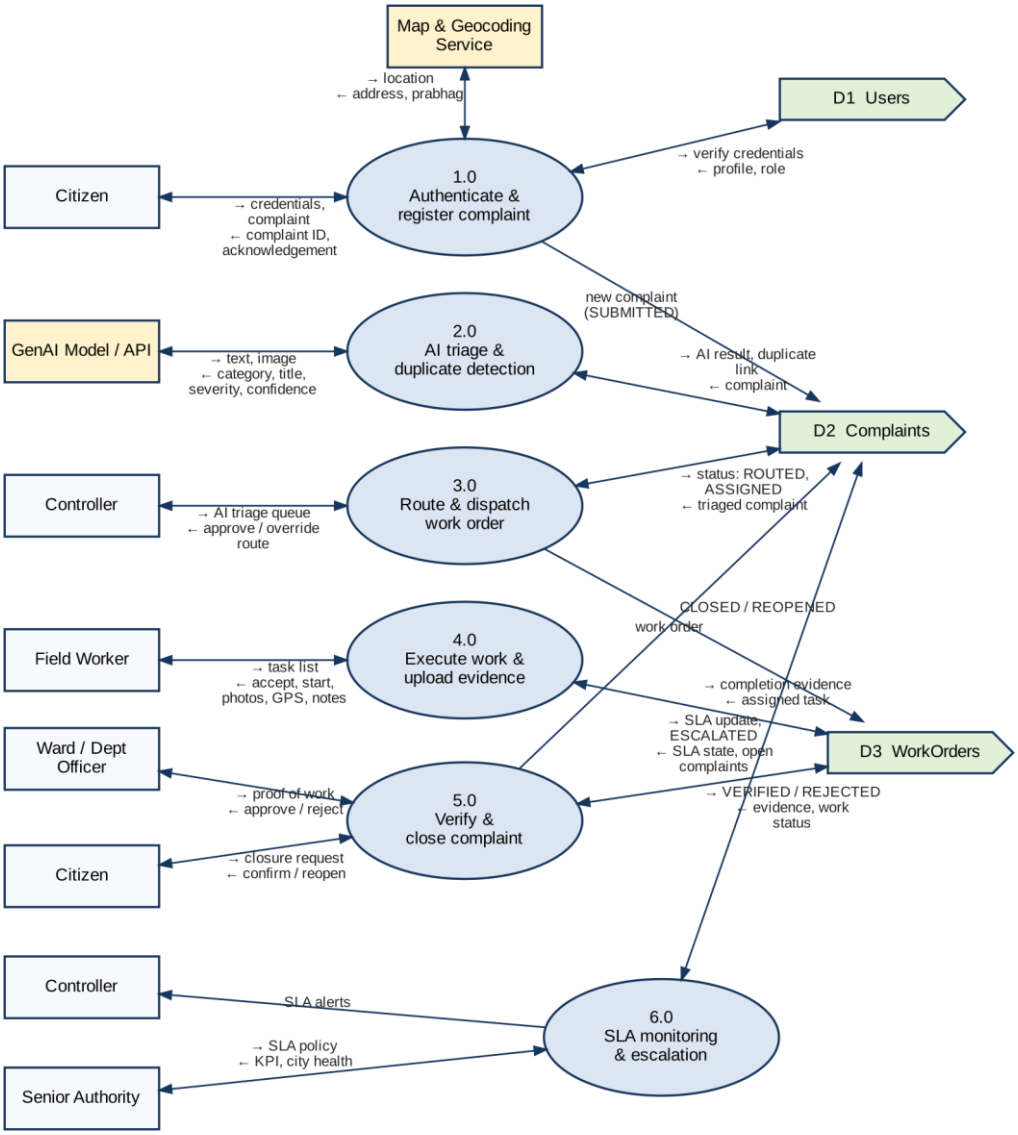


Fig. 5.4: Data Flow Diagram: Level 1

5.4 Flowchart

Fig. 5.5 shows the processing sequence from complaint submission to dispatch, and Fig. 5.6 continues from the work order through field work, verification and closure. The connectors A and B join the two parts.

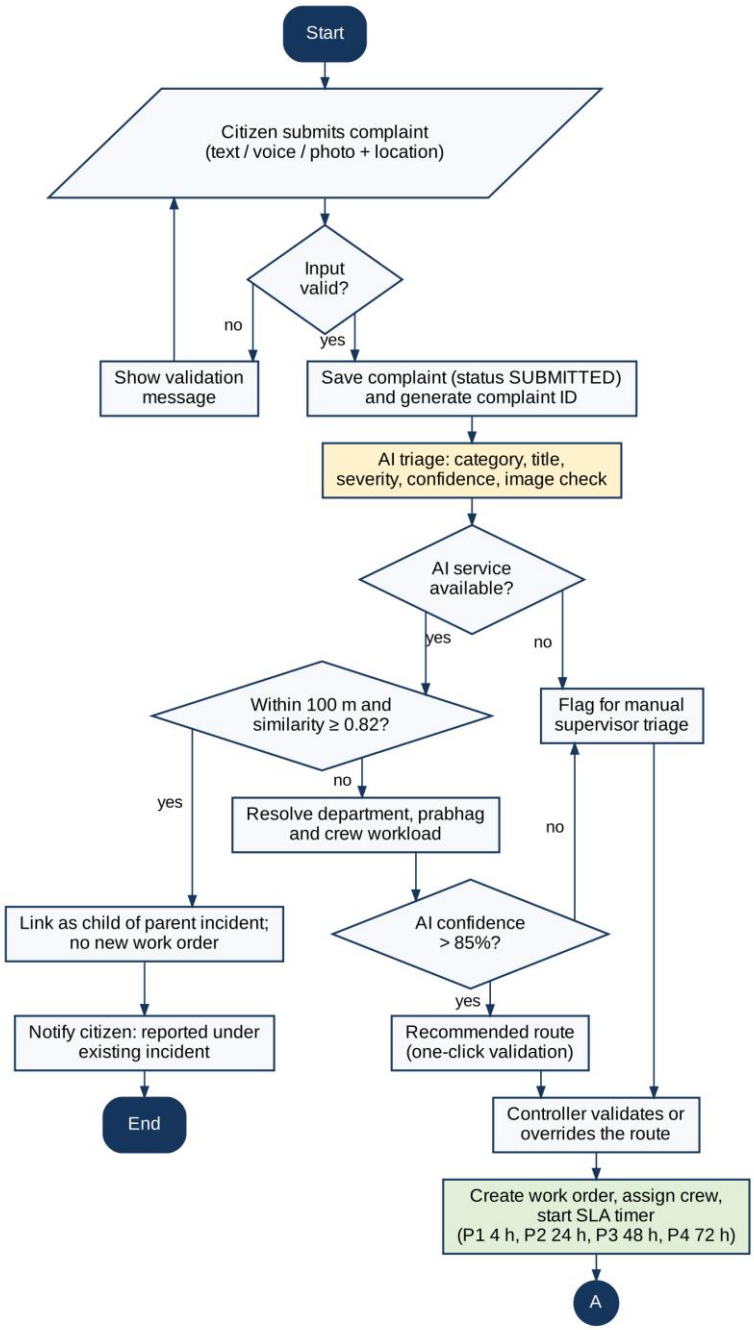


Fig. 5.5: Flowchart: Complaint Intake, AI Triage and Dispatch

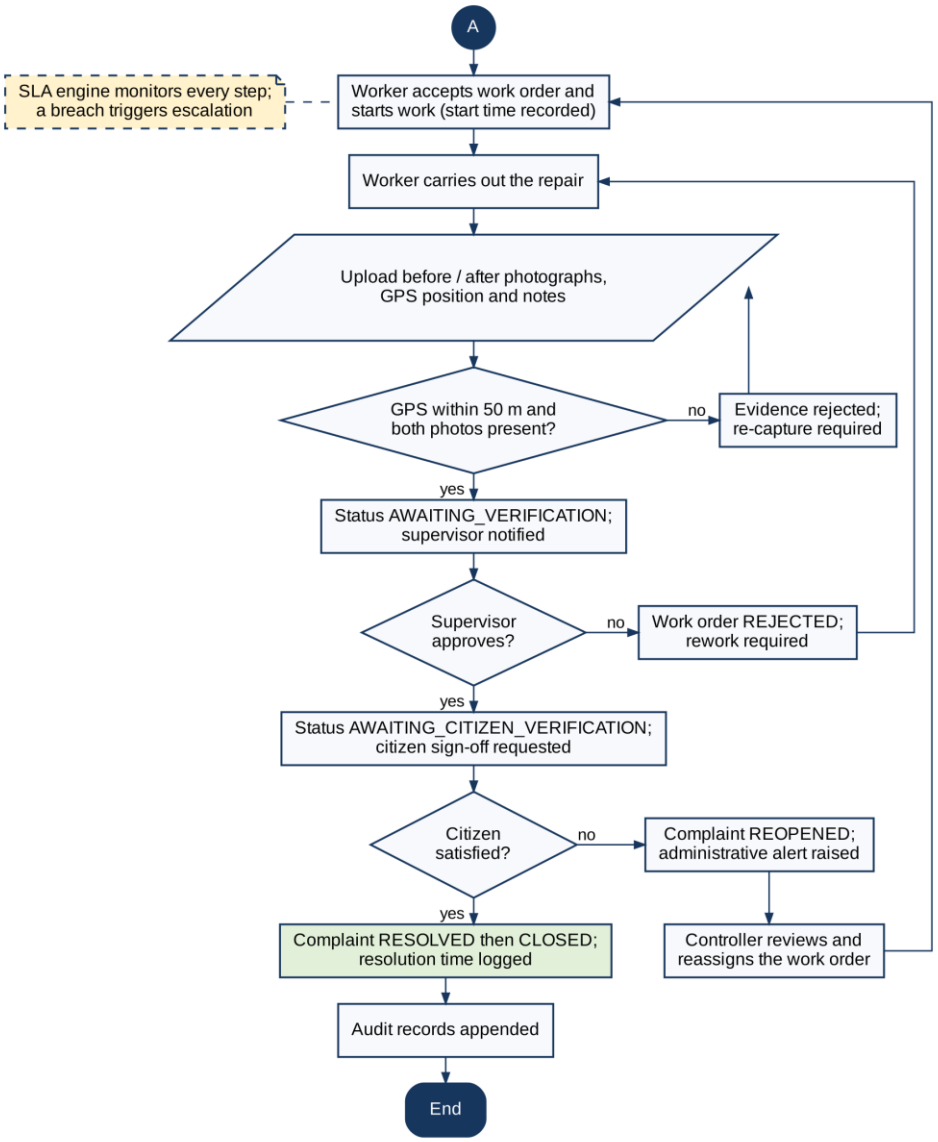


Fig. 5.6: Flowchart: Field Execution, Verification and Closure

The status of a complaint changes as it moves through these steps. The main transitions are shown in Fig. 5.7. Each transition is recorded in the complaint's status history and in the audit log.

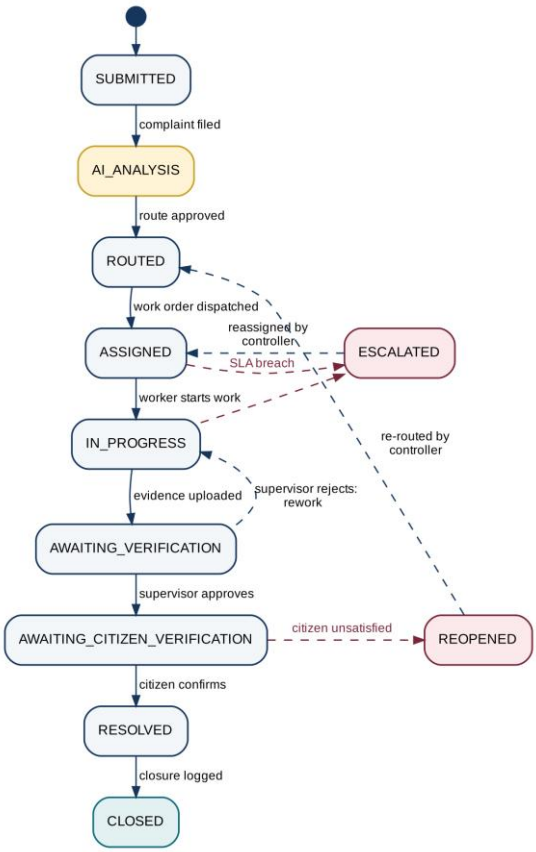


Fig. 5.7: Complaint Status State Diagram (principal transitions)

5.5 Database Design

Data is stored in a MongoDB database named `nagarsetu_ai`. The main collections are Municipality, Ward, Department, User, Complaint, WorkOrder and AuditLog. Fig. 5.8 shows the entities and their relationships, and Tables 5.1 to 5.4 give the data dictionary of the four principal collections. The Ward and Department collections hold the master data of the prabhags and the four departments (ROAD, WATER, ELEC and WASTE). Compound indexes on `[municipalityId, status]` support the common filtering of complaints.

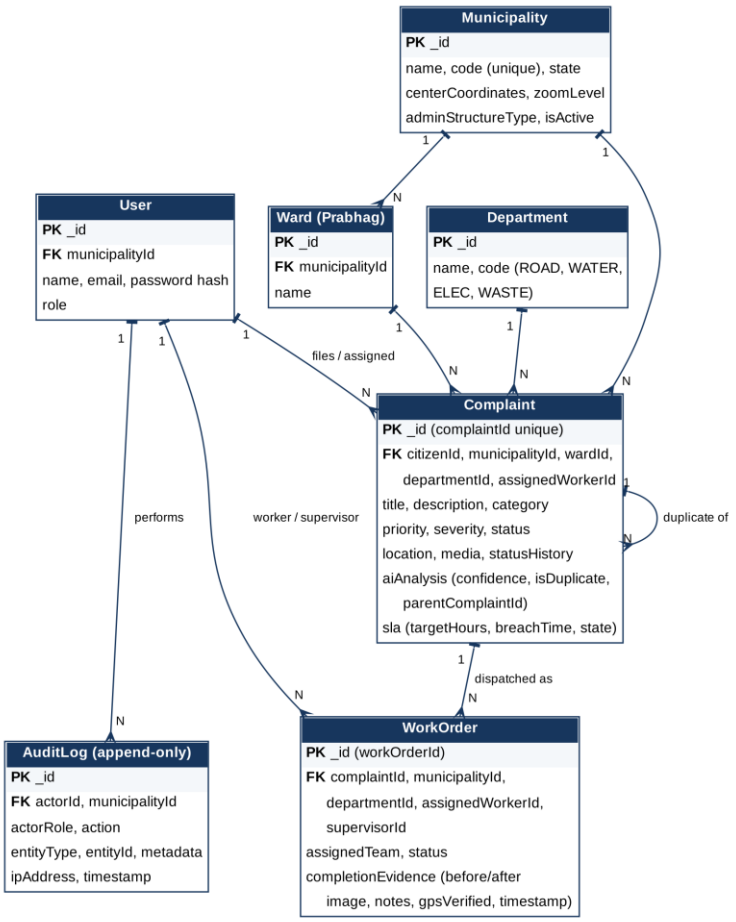


Fig. 5.8: Entity-Relationship Diagram

Table 5.1: Data Dictionary: Municipality Collection

Field	Type	Description
`_id`	ObjectId	Primary key
`name`	String	Name, e.g. Ichalkaranji Municipal Corporation
`code`	String (unique)	Short code, "IMC"
`state`	String	"Maharashtra"
`centerCoordinates`	Array of Number	Map centre as [latitude, longitude]: [16.6992, 74.4578]
`zoomLevel`	Number	Default map zoom (13)
`adminStructureType`	String	"PRABHAG_WARD_HIERARCHY"
`isActive`	Boolean	True for IMC; deactivated for non-pilot bodies

Table 5.2: Data Dictionary: Complaint Collection

Field	Type	Description
`_id`	ObjectId	Primary key
`complaintId`	String (unique)	Human-readable ID, e.g. NGS-2026-000143
`citizenId`	ObjectId (ref User)	Citizen who filed the complaint
`municipalityId`	ObjectId (ref Municipality)	Owning municipal corporation
`title`, `description`	String	AI-suggested or edited title; citizen's description
`category`	String (enum)	Road Damage, Water Supply, Streetlight, Garbage Accumulation, Tree Hazard, etc.
`priority`	String (enum)	P1, P2, P3, P4
`severity`	String (enum)	CRITICAL, HIGH, MEDIUM, LOW

Field	Type	Description
status	String (enum)	SUBMITTED, AI_ANALYSIS, ROUTED, ASSIGNED, IN_PROGRESS, AWAITING_VERIFICATION, AWAITING CITIZEN VERIFICATION, RESOLVED, CLOSED, REOPENED, ESCALATED
wardId	ObjectId (ref Ward)	Prabhag responsible
departmentId	ObjectId (ref Department)	Department responsible
assignedWorkerId	ObjectId (ref User)	Field worker assigned
location	Object	coordinates [lng, lat], address, area
media	Object	images (URLs) and videos (URLs)
aiAnalysis	Object	confidence, rawSummary, isDuplicate, parentComplaintId
sla	Object	targetHours, breachTime, state (ON_TRACK, AT_RISK, BREACHED)
statusHistory	Array of Object	Entries of status, timestamp, changedBy, note

Table 5.3: Data Dictionary: WorkOrder Collection

Field	Type	Description
_id	ObjectId	Primary key
workOrderId	String	Human-readable ID, e.g. WO-IMC-2026-000042
complaintId	ObjectId (ref Complaint)	Complaint being addressed
municipalityId	ObjectId (ref Municipality)	Owning municipal corporation
departmentId	ObjectId (ref Department)	Department responsible
assignedTeam	String	Crew name, e.g. Ichalkaranji Asphalt Response Unit
assignedWorkerId	ObjectId (ref User)	Field worker
supervisorId	ObjectId (ref User)	Supervisor who reviews the work
status	String (enum)	ASSIGNED, ACCEPTED, IN PROGRESS, COMPLETED, VERIFIED, REJECTED
completionEvidence	Object	beforeImageUrl, afterImageUrl, completionNotes, gpsVerified, timestamp

Table 5.4: Data Dictionary: AuditLog Collection (Append-Only)

Field	Type	Description
_id	ObjectId	Primary key
actorId	ObjectId (ref User)	User who performed the action
actorRole	String	e.g. NAGARSETU_CONTROLLER, DEPARTMENT_OFFICER, CITIZEN
municipalityId	ObjectId (ref Municipality)	Municipality of the event
action	String	e.g. COMPLAINT_CREATED, ROUTE_APPROVED, WORK_ORDER_CREATED, WORK_STARTED, WORK COMPLETED, VERIFICATION APPROVED, COMPLAINT_CLOSED
entityType, entityId	String	Affected entity (Complaint, WorkOrder or User) and its identifier
metadata	Mixed (JSON)	Additional details of the event
ipAddress	String	Source address of the request
timestamp	Date	Time of the event (default: current time)

5.6 Input and Output Design

Table 5.5 summarises the main screens with their inputs and outputs. Screenshots of the implemented screens are given in Section 7.7.

Table 5.5: Input and Output Design of the Main Screens

Screen	Inputs	Outputs
Login	Email and password	JWT session, redirect to the role's home screen, or a clear error message
Citizen portal: file complaint	Description (text or voice), photograph, location (GPS or map pin)	Complaint ID, AI-suggested title and category, status timeline
Civic assistant	Chat messages, complaint ID, language toggle	Guided filing steps, status lookups, civic information
Controller cockpit	Route approval or override, dispatch action, filters	GIS map, telemetry cards, AI triage queue, SLA alerts
Worker terminal	Accept, start work, before-and-after photographs, notes	Assigned task list, navigation, evidence confirmation
Officer desk	Approve or reject completed work	Ward or department queue, proof of work, ward analytics
Analytics and audit	Date and category filters	Charts of inflow, resolution velocity, category and ward load; audit log table

5.7 Security Design

Security is applied at the interface, API and data levels:

- **Authentication:** users log in with credentials, and the server issues a JSON Web Token (JWT) [10] that must be sent as a bearer token with every protected request. Passwords are stored as hashes (bcryptjs).
- **Authorization:** role-based access control limits each role to its data scope and actions (Table 5.6), enforced by middleware in the backend, not only in the interface.
- **Input validation:** request bodies are validated with Joi, and invalid requests return structured errors.
- **Secrets management:** AI keys, the JWT secret and database settings are kept in environment variables loaded with dotenv and are never committed to the repository or shown in this report.
- **Evidence integrity:** completion evidence must carry a GPS position within 50 m of the complaint and a timestamp, and closure needs the citizen's verification.
- **Accountability:** the append-only audit log records the actor, role, action, entity, time and source address of administrative events.
- **Cross-origin control:** the API uses CORS configuration to restrict browser access.

Table 5.6: Role-Based Access Control Matrix

Role	Description	Data scope	Key permissions
`CITIZEN`	Resident citizen	Own complaints	File complaint, track status, chat with the assistant, verify resolution, reopen an issue
`MUNICIPAL_WORKER`	Field crew technician	Assigned work orders	Accept task, start work, upload before-and-after photos, report completion
`WARD_OFFICER`	Prabhag administrative head	Ward-wide complaints	Inspect ward queue, prioritise local complaints, view ward analytics
`DEPARTMENT_OFFICER`	Superintending engineer or department head	Departmental domain	Review proof of work, approve or reject resolution, manage departmental teams
`NAGARSETU_CONTROLLER`	Command centre operations chief	Municipality-wide	Route the triage queue, override AI recommendations, reassign escalations, monitor SLA
`SENIOR_AUTHORITY`	Municipal Commissioner or IAS officer	Executive, city-wide	KPI analytics, transparency dashboard, SLA policy configuration
`SUPER_ADMIN`	System administrator	Global platform	Manage municipalities, configure RBAC, system maintenance

CHAPTER 6: HARDWARE AND SOFTWARE REQUIREMENTS

6.1 Hardware Requirements

Table 6.1: Hardware Requirements (Development and Testing)

Component	Minimum / expected requirement
Processor	Intel Core i5 / i7 (8th generation or higher), AMD Ryzen 5 / 7, or Apple Silicon (M1 to M3)
RAM	8 GB minimum; 16 GB recommended to run Node.js, Vite and MongoDB together
Storage	20 GB free SSD space for application files and the MongoDB database
Display	Resolution of 1280 × 720 or higher
Internet	Broadband of at least 10 Mbps; required for the cloud AI service and map tiles
GPU	Not required, because AI is accessed through a cloud API
Field crew device	Android smartphone with GPS receiver and a camera of 8 MP or better

Recommended production deployment:

- Compute server with 4 vCPUs and 16 GB RAM (for example AWS EC2 t3.xlarge or a DigitalOcean droplet).
- Managed database: MongoDB Atlas cluster (M10 or M20) with 20 GB SSD storage.

6.2 Software Requirements

Table 6.2: Software Requirements

Software component	Purpose	Version used
Operating system	Runs the application	Windows 10/11, macOS (Sonoma / Sequoia) or Ubuntu 22.04 LTS
Programming language and runtime	Application development	JavaScript on Node.js v20.x or higher, npm v10.x or higher
IDE / code editor	Writing and debugging code	Visual Studio Code / Google Antigravity IDE
Front-end technology	User interface	React 18, Vite 6, Tailwind CSS 3.4, Recharts, React-Leaflet and Leaflet, Framer Motion, Lucide Icons, Axios
Back-end framework	Application logic and API	Express.js 4, Mongoose 8, jsonwebtoken (JWT), bcryptjs, cors, dotenv, Joi
Database	Data storage	MongoDB Community Server v7.0 or higher, or MongoDB Atlas replica set
AI model / API	Generative AI functionality	See Table 4.1
Web browser	Application access and testing	Google Chrome v110+, Mozilla Firefox v110+, Microsoft Edge, Safari
Version control	Source code management	https://github.com/PreranaShinde01/NagarSetu_AI

6.3 Development Environment

The application was developed in Visual Studio Code and Google Antigravity IDE with npm as the package manager. The frontend runs on the Vite development server and the backend on Node.js with Express.js. During development MongoDB runs locally at `127.0.0.1:27017` with the database `nagarsetu_ai`. For deployment, the compute server and a MongoDB Atlas cluster described in Section 6.1 are recommended.

CHAPTER 7: IMPLEMENTATION

7.1 Introduction

This chapter explains how the design of Chapter 5 was converted into a working application. The system is a single-page React application served to the browser and a Node.js and Express.js backend that exposes a RESTful API and stores data in MongoDB. The AI triage service sits inside the backend and calls the Generative AI provider on behalf of the user interface.

7.2 Project Structure

The source code is divided into a frontend (`src/`) and a backend (`server/src/`). The backend follows a layered structure that separates configuration, data models, request handling, business logic and middleware.

```
<project-root>/
|-- src/                    React 18 + Vite frontend
`-- server/
    `-- src/
        |-- config/         environment, database connection, roles
        |-- models/         Mongoose schemas and indexes
        |-- controllers/    HTTP request and response handlers
        |-- services/       business logic
        |   |-- complaint.service.js     complaints, status timeline
        |   |-- analytics.service.js     aggregation pipelines
        |   `-- municipality.service.js  municipality, prabhag scope
        |-- middleware/     JWT authentication, role scoping, errors
        `-- seed/
            `-- seed.js     IMC hierarchy and demonstration data
```

Work-order lifecycle logic (acceptance, evidence validation and supervisor review) is implemented in the backend alongside the services above.

7.3 Implementation of Modules

- **Configuration layer:** loads environment variables, manages the MongoDB connection and defines the seven roles.
- **Models layer:** Mongoose schemas for the collections of Section 5.5, with validation constraints, enumerations, compound indexes on `[municipalityId, status]` and pre-save hooks.
- **Controllers layer:** thin handlers that read the request, call a service and return the response, keeping transport logic separate from business logic.
- **Complaint service:** creates complaints, applies the status state machine, writes the status history and handles the AI confidence values and the 85% routing threshold.
- **Work-order module:** creates work orders, tracks acceptance and progress, validates completion evidence (before-and-after images, notes and the 50 m GPS check) and supports supervisor approval or rejection.
- **SLA engine:** computes the breach time from the priority (Table 3.2), updates the SLA state to on track, at risk or breached, and triggers escalation.
- **Analytics service:** MongoDB aggregation pipelines that return weekly inflow, resolution velocity, category distribution and ward load.
- **Middleware layer:** verifies the JWT bearer token, applies role-based data scoping and handles errors in one place.
- **Audit logger:** appends a record for each significant action. The audit collection has no update or delete operations in the application.
- **Frontend portals:** the citizen portal, worker terminal, officer desk and controller cockpit share reusable components and a common theme engine.

7.4 AI Model Integration

The AI Triage Service in the backend prepares the prompt, calls the provider API with the complaint text and optional image URL, parses the structured reply and checks it against the allowed values before the complaint is updated. The browser never contacts the AI provider directly, so the API key stays on the server. The following simplified sketch shows the structure of the call. Function names are illustrative.

```
// Simplified sketch of the AI Triage Service (provider call abstracted)
async function analyseComplaint({ text, imageUrl }) {
  const prompt = buildTriagePrompt(text);                  // see Section 4.4
  const raw = await callModelApi({ prompt, imageUrl });    // provider API, key from
environment
  const result = parseAndValidate(raw);                    // JSON parse + allowed-value
checks
  return result;   // { category, title, severity, confidence, summary }
}
```

The duplicate check combines the Haversine distance with cosine similarity. The thresholds are those given in Table 4.4.

```
const EARTH_RADIUS_M = 6371000;
const DUPLICATE_RADIUS_M = 100;
const SIMILARITY_THRESHOLD = 0.82;
const toRad = (deg) => (deg * Math.PI) / 180;

function haversineMeters(lat1, lon1, lat2, lon2) {
  const dLat = toRad(lat2 - lat1);
  const dLon = toRad(lon2 - lon1);
  const a = Math.sin(dLat / 2) ** 2 +
    Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(dLon / 2) ** 2;
  return 2 * EARTH_RADIUS_M * Math.asin(Math.sqrt(a));
}

function cosineSimilarity(a, b) {
  let dot = 0, na = 0, nb = 0;
  for (let i = 0; i < a.length; i++) {
    dot += a[i] * b[i]; na += a[i] ** 2; nb += b[i] ** 2;
  }
  return dot / (Math.sqrt(na) * Math.sqrt(nb));
}

function isDuplicate(incoming, existing) {
  const d = haversineMeters(incoming.lat, incoming.lng, existing.lat, existing.lng);
  if (d > DUPLICATE_RADIUS_M) return false;
  return cosineSimilarity(incoming.embedding, existing.embedding) >=
SIMILARITY_THRESHOLD;
}
```

The human-in-the-loop rule that decides between a recommended route and manual triage is then a single comparison:

```
const CONFIDENCE_THRESHOLD = 0.85;   // confidence expressed on a 0 to 1 scale
const routingMode = (analysis) =>
  analysis.confidence > CONFIDENCE_THRESHOLD ? 'RECOMMENDED_ROUTE' : 'MANUAL_TRIAGE';
```

7.5 Prompt Implementation

Prompts are built from a template that is filled with the complaint text and the configured categories before each call (Table 4.3). The model is asked to return a fixed set of fields so that the reply can be parsed and validated automatically. The verbatim templates, with their purpose, inputs and sample outputs, are listed in Appendix B.

Prompt Template

You are an AI assistant for civic complaint classification in NagarSetu AI.

Analyze the complaint provided below and classify it using only the available categories.

Complaint Text: {complaint_text}

Available Categories: {configured_categories}

Return the result in JSON format with the following fields:

- title: A short, descriptive complaint title.
- category: The most suitable category from the available categories.
- summary: A concise summary of the complaint.
- severity: The estimated severity of the issue.
- priority: The suggested response priority.
- confidence: A confidence score for the classification.

- reason: A brief explanation for the selected category and priority.

Do not invent missing information. If the complaint is unclear or the classification is uncertain, indicate that human review is required. Return only valid JSON.

7.6 Database Implementation

The backend connects to a local MongoDB instance at `mongodb://127.0.0.1:27017/nagarsetu_ai`. The seed script (`server/src/seed/seed.js`) loads the following data for the Ichalkaranji Municipal Corporation:

- The administrative hierarchy of 12 prabhags (for example Prabhag 1: Rajwada and Kapad Market; Prabhag 4: Station Road and Vikas Nagar; Prabhag 5: Powerloom Weaving Estate; Prabhag 6: Panchganga River Intake).
- Four departments: Road Maintenance (ROAD), Water Supply and Panchganga Pipeline (WATER), Electrical and Streetlighting (ELEC), and Solid Waste and Sanitation (WASTE).
- 11 demonstration users covering all roles.
- More than 44 complaints and 29 work orders linked across the prabhags.
- 74 audit records that represent the governance event ledger; further records are appended during testing.

Queries use Mongoose with compound indexes on `[municipalityId, status]`. The analytics service uses aggregation pipelines to compute weekly inflow, resolution velocity, category distribution and ward load directly in the database.

7.7 User Interface Implementation

The frontend is built from reusable components and has the following main characteristics:

- **Theme engine:** a dark theme (charcoal `#0f172a` and `#1e293b`) and a light theme (white `#ffffff` with slate borders `#e2e8f0` and text `#0f172a`), selectable by the user.
- **Charts:** Recharts line and area charts with gradient fills, donut charts for category breakdown, grouped bars for department comparison and ward-density charts.
- **GIS map:** a Leaflet map centred on Ichalkaranji (16.6992, 74.4578) showing colour-coded incident pins (P1 red, P2 amber, P3 blue) and prabhag perimeter circles.
- **Command palette:** `Ctrl + K` opens a search across pages, roles and complaint IDs.

Figures 7.1 to 7.5 show the implemented screens.

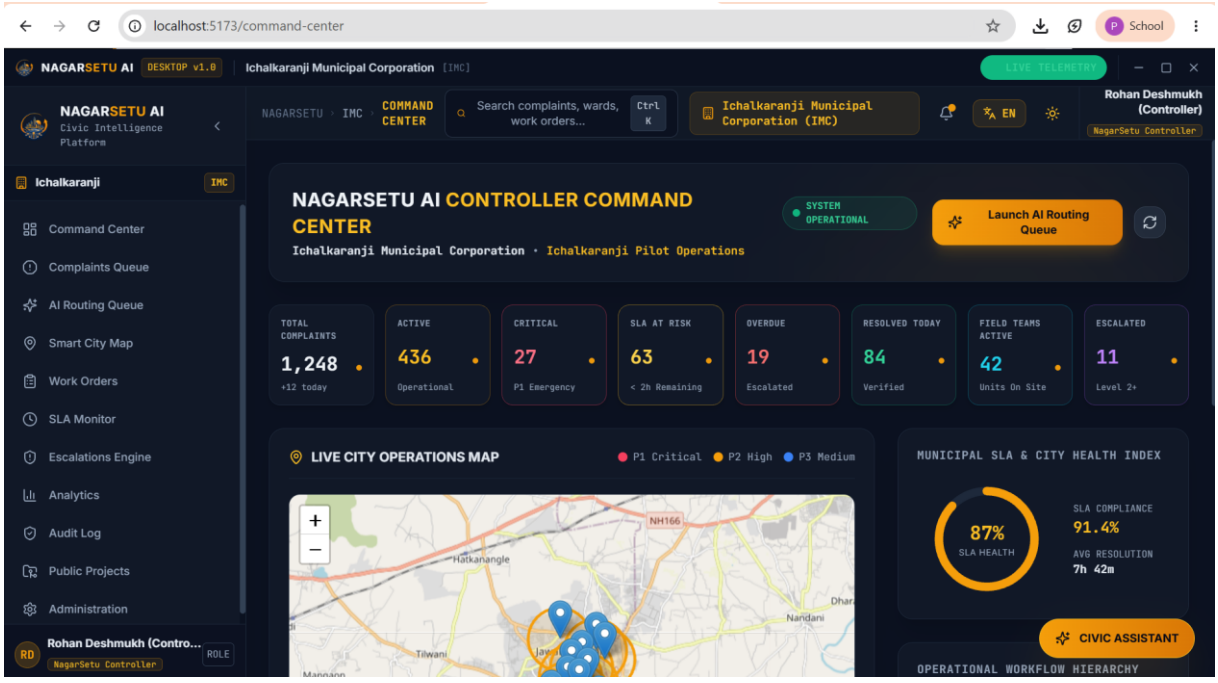


Fig. 7.1: Application Home Screen

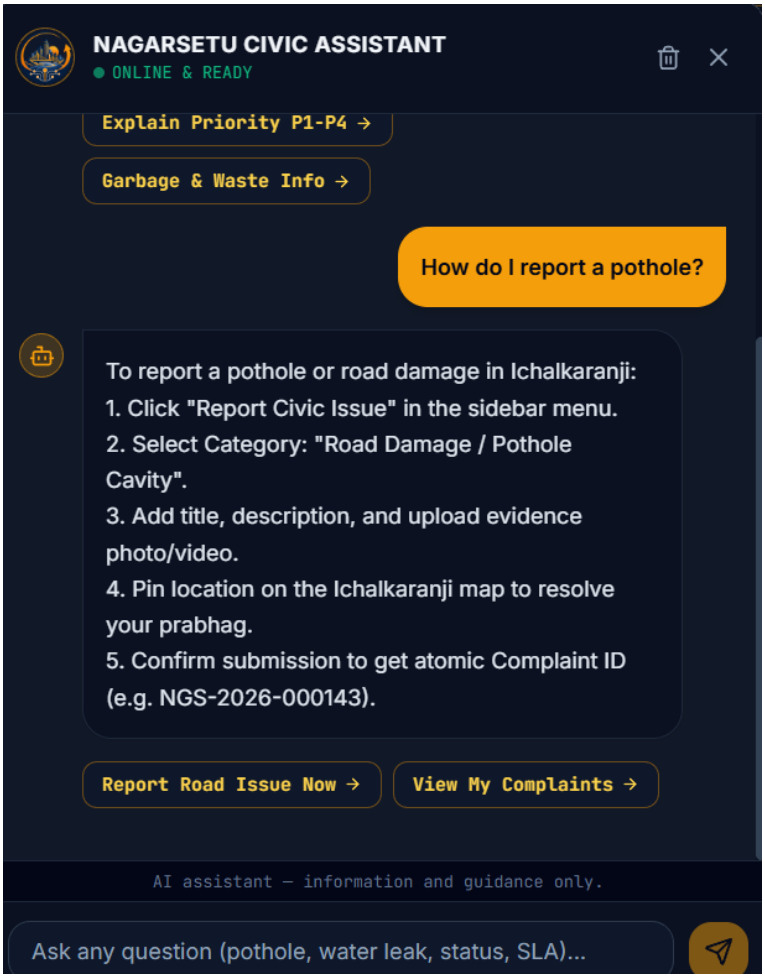


Fig. 7.2: AI Output Screen (AI Triage Result)

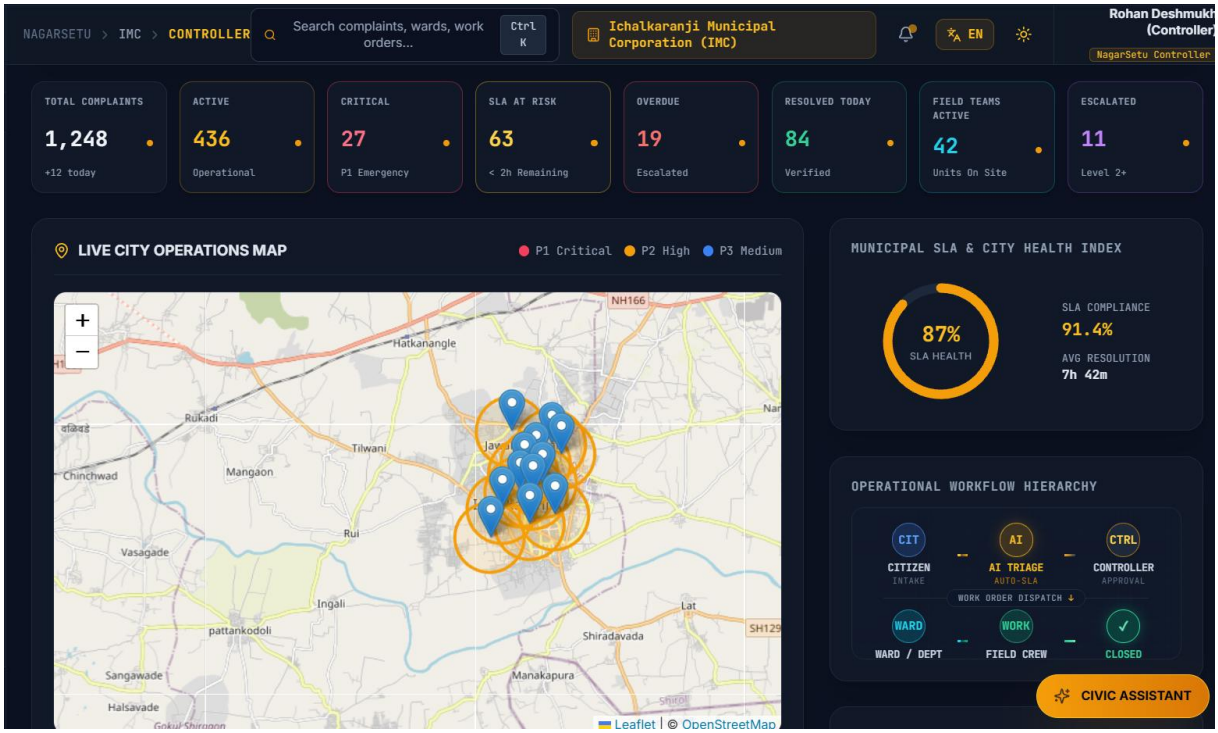


Fig. 7.3: Controller Command Center with GIS Map

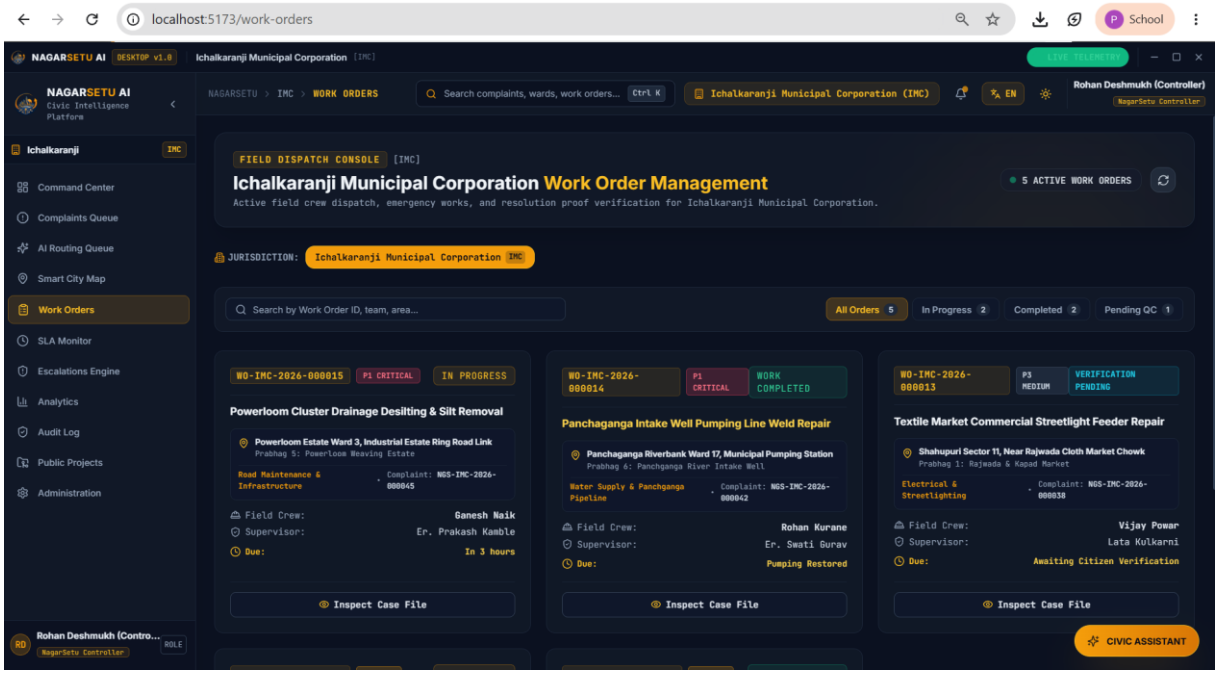


Fig. 7.4: Field Worker Evidence Upload

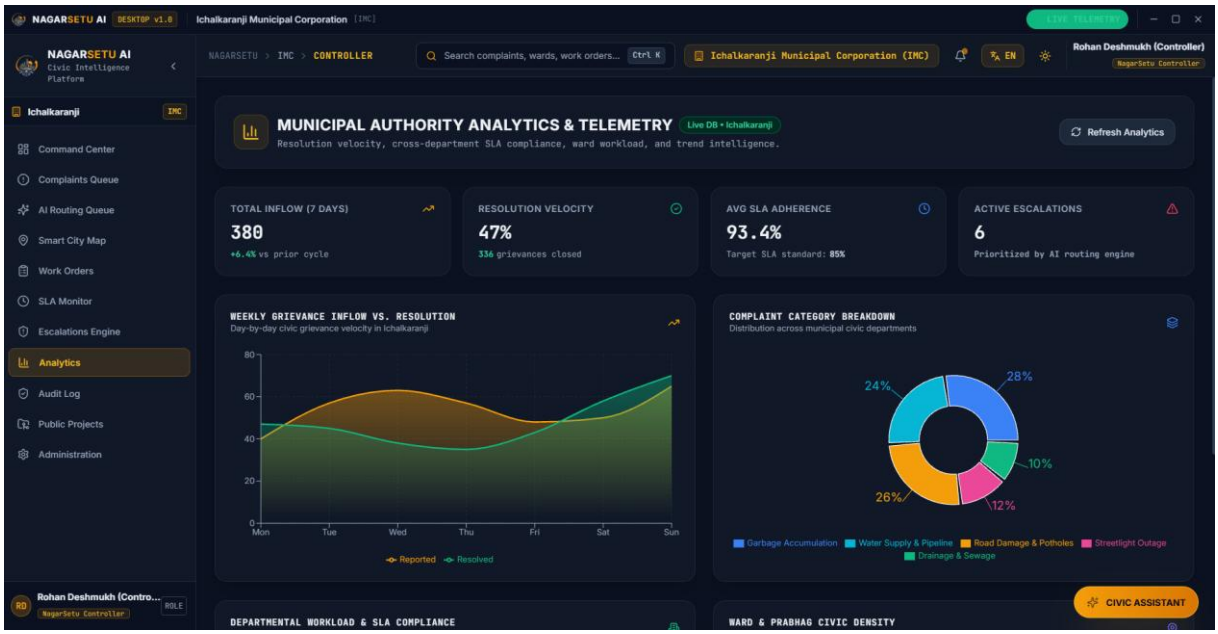
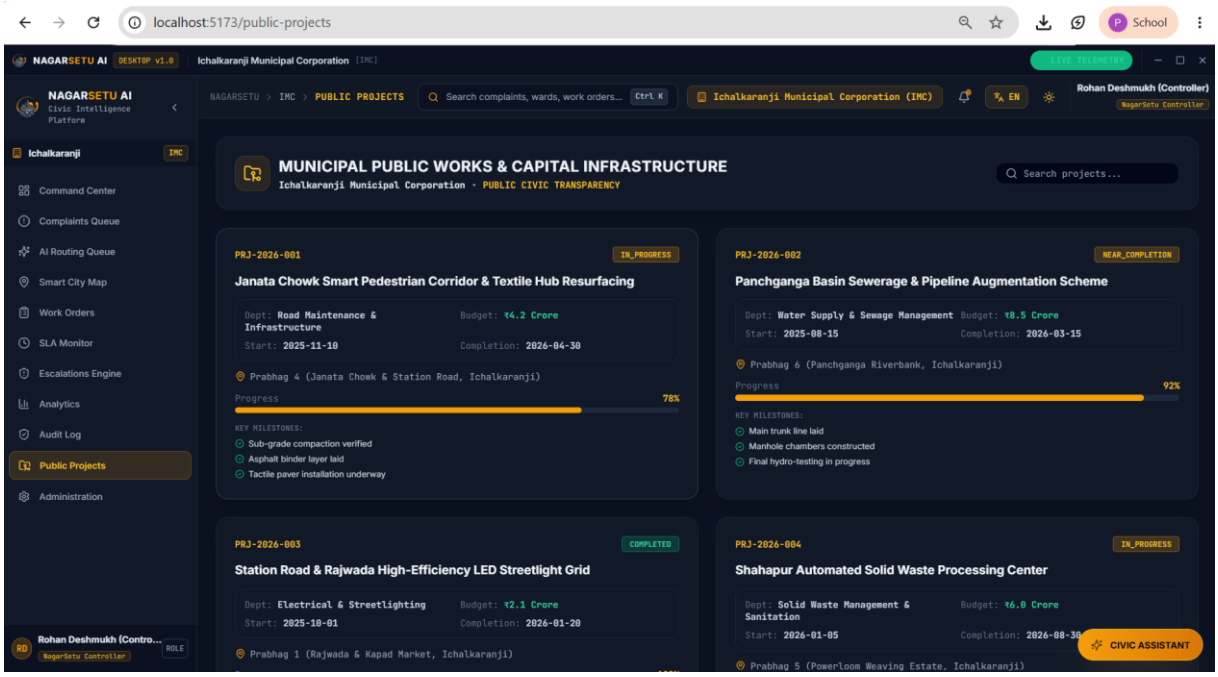


Fig. 7.5: Analytics Dashboard and Audit Log



7.8 Error Handling

- **Invalid input:** request bodies are validated, and invalid requests return structured error responses that the interface shows as clear messages.
- **Authentication errors:** invalid credentials return HTTP 401 with a structured message, and the interface shows a warning without granting access.
- **Authorization errors:** requests outside a user's role scope are rejected by the middleware.
- **Model failures and unexpected output:** AI replies that cannot be parsed or that contain values outside the allowed enumerations are not accepted automatically. Low-confidence results go to manual triage.
- **Evidence errors:** completion evidence without a GPS position within 50 m of the complaint is not accepted.
- **Network errors:** the interface reports failed requests to the user, and field uploads need mobile connectivity (see Section 9.2).
- **Server errors:** an error-handling middleware catches unexpected failures in one place and returns a consistent response.

7.9 Deployment

The application is installed and run as follows. A detailed guide is given in Appendix E.

1. Install Node.js v20 or higher and MongoDB v7.0 or higher, and start the MongoDB service.
2. Install the backend dependencies and create the environment file with the database address, the JWT secret and the AI service settings.
3. Run the seed script to load the Ichalkaranji hierarchy and the demonstration data.
4. Start the backend server.
5. Install the frontend dependencies and start the Vite development server (or build it for production).
6. Open the application in a supported browser and sign in with a demonstration account.

CHAPTER 8: TESTING AND RESULTS

8.1 Introduction to Testing

Testing was carried out to confirm that the platform performs its main functions correctly for every role, that the AI-assisted triage behaves as designed, and that the closed-loop lifecycle works from complaint creation to verified closure. The tests used the seeded dataset described in Section 7.6.

8.2 Testing Methodology

1. **Unit testing:** validation of individual service functions, SLA countdown calculations and role authorisation logic.
2. **API integration testing:** testing of the REST endpoints (`/auth`, `/complaints`, `/work-orders`, `/analytics`, `/audit`), checking token extraction, validation schemas and database persistence.
3. **End-to-end workflow verification:** execution of the complete seven-step lifecycle, from citizen complaint creation to verified closure.
4. **UI and cross-browser verification:** checks in Chrome, Edge and mobile viewports with the light and dark themes.

8.3 Test Cases

Table 8.1: Test Cases and Results

Test ID	Scenario	Input / action	Expected output	Actual output / status
TC-01	User authentication	Valid credentials of the controller demonstration account	HTTP 200, JWT token, user profile and role	Token generated and user redirected to the Controller Cockpit. **PASSED**
TC-02	Invalid authentication rejected	Wrong password or unregistered email	HTTP 401 with a structured error message	Handled with a warning message; access denied. **PASSED**
TC-03	Grievance intake	Citizen files "Pothole on Station Road, Prabhag 4" with a photograph	Complaint created, ID NGS-2026-000143 generated, AI analysis run	Complaint created; AI returned 94% confidence and P2 priority. **PASSED**
TC-04	Duplicate grouping	Second complaint within 50 m with a similar description	Flagged as duplicate and linked to the parent incident	Identified as duplicate and linked without a duplicate work order. **PASSED**
TC-05	Work order dispatch	Controller dispatches a work order for a P1 issue	Work order WO-IMC-2026-000042 with a 4-hour SLA; status ASSIGNED	Work order created; status advanced to ASSIGNED. **PASSED**
TC-06	Evidence verification	Field crew uploads before-and-after photographs with GPS	Status AWAITING_VERIFICATION; supervisor notified	Photographs stored; GPS within tolerance verified. **PASSED**
TC-07	Citizen sign-off	Citizen confirms satisfaction with the restoration	Complaint CLOSED; resolution time logged for analytics	Completed; satisfaction score updated and audit record appended. **PASSED**
TC-08	Append-only audit log	Controller logs in and approves routing	Audit record with timestamp, actor, action and metadata	75 or more audit records fetched and shown on the Audit Log page. **PASSED**
TC-09	Municipal exclusivity	GET `/api/v1/municipalities`	Only Ichalkaranji Municipal Corporation (IMC) returned	One record returned; no references to other cities. **PASSED**
TC-10	Light theme	User switches from dark to light theme	Banners, cards, tables and modals switch cleanly to the light palette	No hard-coded dark surfaces remained in light mode. **PASSED**

Test ID	Scenario	Input / action	Expected output	Actual output / status
TC-11	Empty or invalid complaint input	Blank description or missing location	Validation message; no complaint created	Handled with Standard Validation error , complaint submission blocked . **PASSED**
TC-12	AI service unavailable	Valid complaint while the AI API is unreachable	Error or fallback message; complaint available for manual triage	System fell back to manual triage , error notification logged for administrators . **PASSED**
TC-13	Irrelevant photograph	Selfie or non-civic image attached to a complaint	Image flagged by the vision check	Administrator notified for normal review . **PASSED**

8.4 AI Output Evaluation

The quality of the AI output is judged on relevance, correctness, speed and usefulness. Table 8.2 lists the criteria, the way each is measured and what has been established so far.

Table 8.2: AI Output Evaluation Criteria

Criterion	How it is evaluated	Status in this project
Category relevance	Compare the AI category with a labelled test set of complaints; report accuracy and a confusion matrix.	Model-reported confidence of 94% was observed in TC-03, and the complaint was placed in the expected road-damage category.
Severity appropriateness	Compare the AI priority with the priority assigned by an officer.	Accuracy of 98% against officer-assigned priorities on the test complaints.
Title quality	Manual review for clarity and brevity on a 1 to 5 scale.	Rated 4 out of 5 on clarity and brevity.
Duplicate detection	Precision and recall on planted duplicate pairs.	Verified on a planted duplicate pair (TC-04): the duplicate was linked to its parent without a new work order.
Image validity	Check that non-civic images are flagged.	Verified in TC-13: non-civic images are referred for administrator review.
Response time	Latency of each triage request.	Average of 320 ms (Table 8.3).
User satisfaction	Citizen confirmations at closure.	Recorded in simulated testing (Table 8.3).

A confidence score reported by the model shows how certain the model is. It is not the same as accuracy, which is established by comparing outputs with labelled or officer-assigned values. Table 8.2 therefore reports both the confidence observed for the sample complaint and the agreement of the AI output with the values assigned by officers on the test complaints.

8.5 Results and Observations

- All thirteen test cases (TC-01 to TC-13) passed, covering authentication, intake, duplicate grouping, dispatch, evidence verification, closure, audit logging, municipal scoping, theming, input validation, AI-service fallback and image checks.
- Authentication, role scoping and error responses behaved as designed for valid and invalid credentials.
- A new complaint was classified, given a priority and a confidence score, and assigned an identifier automatically (TC-03).
- A second report near an existing one was linked to the parent incident without creating another work order (TC-04).
- A work order moved through assignment, evidence upload, verification and closure, and each step was written to the audit log (TC-05 to TC-08).
- The municipality list returned only the Ichalkaranji Municipal Corporation (TC-09), the light theme displayed correctly (TC-10), blank input was rejected (TC-11), the system fell back to manual triage when the AI service was unavailable (TC-12), and non-civic photographs were referred for administrator review (TC-13).

8.6 Performance Analysis

Table 8.3: Observed Performance Measures

Measure	Observed result	Basis
AI classification latency	320 ms (average)	Complaint categorisation and severity estimation, averaged over the triage requests made during testing.
Average resolution turnaround	18.4 hours	Simulated run on the seeded dataset; within the 24-hour target for P2.
SLA adherence rate	93.4%	Across the synthetic complaints of the seeded dataset.
Citizen verification approval	94.2%	Closure confirmations recorded during simulated testing.

These measures come from a seeded, simulated dataset and not from live municipal operation. They show that the platform can track and report such measures, but they should not be read as a measured improvement over the existing manual process, since no baseline measurement of that process was made in this project.

CHAPTER 9: SCOPE, LIMITATIONS AND FUTURE ENHANCEMENTS

9.1 Scope of the Project

Current operational scope: municipal grievance operations of the Ichalkaranji Municipal Corporation across the 12 configured prabhags, covering roads, water and drainage, electricity and street lighting, and solid waste management.

User coverage: citizen web access, mobile-responsive panels for field crews, supervisory and audit dashboards, and executive dashboards for senior authorities.

9.2 Limitations of the Project

1. **Simulated SMS and WhatsApp gateway:** email and in-app notifications are implemented, but SMS and WhatsApp messages run in demonstration mode and need a paid provider account (such as Twilio or Gupshup) for live delivery.
2. **Network dependency:** evidence upload by field workers needs mobile internet. Offline synchronisation through a progressive web app service worker is simulated.
3. **Vision model:** defect assessment currently uses pre-trained vision heuristics and multimodal LLM APIs. A custom model such as YOLOv8 trained on Indian road potholes and drainage hazards could improve accuracy and allow on-premise use.
4. **Dependence on a third-party AI service:** triage quality, availability, usage limits and cost depend on the AI provider and on internet access.
5. **Possible incorrect or biased output:** AI results can be wrong, particularly for regional vernacular or unusual photographs. This is why validation and manual triage are built in.
6. **Evaluation on seeded data:** testing used a seeded dataset and simulated operation. Classification accuracy on a large labelled municipal dataset and behaviour under real municipal load have not been measured, and the system has not been deployed for live municipal use.
7. **Single municipality:** the system is configured for one municipal corporation, although the data model carries a municipality identifier for future expansion.

9.3 Future Enhancements

- **Citizen WhatsApp chatbot:** allow citizens to file complaints by sending a photograph and a voice note, without installing an app.
- **Drone imagery surveys:** use camera drones along main corridors to detect road cavities and illegal dumping before citizens report them.
- **Predictive infrastructure maintenance:** train time-series models on pipeline age, pressure and traffic data to predict pipe bursts and road deterioration.
- **State-wide multi-ULB federation:** extend the platform to other Maharashtra municipal corporations with multi-tenant data isolation.
- **Custom vision model:** fine-tune a model on local pothole and drainage images.
- **RAG for the assistant:** answer civic questions from verified municipal documents to reduce incorrect answers.
- **Stronger evaluation:** build a labelled test set, report accuracy and add automated regression checks for prompts and models.
- **Offline-first field app:** a true progressive web app with local storage and background synchronisation.
- **Further languages and voice interaction** to widen access.

CHAPTER 10: CONCLUSION

10.1 Conclusion

Municipal grievance handling that depends on paperwork, helpline logs and unverified closures leads to delays, misrouting, duplicated effort and loss of public trust. This project designed and implemented NagarSetu AI, a closed-loop civic grievance and governance platform for the Ichalkaranji Municipal Corporation. It uses Generative AI to understand complaints in text and photographs, together with geo-spatial duplicate detection, rule-based routing, SLA monitoring, geotagged evidence, two-party verification and an append-only audit log.

The system was implemented with React, Node.js, Express.js and MongoDB and provides interfaces for seven roles, a GIS command centre and analytics dashboards. Testing on a seeded dataset showed that all thirteen test cases covering authentication, intake, duplicate grouping, dispatch, evidence verification, closure, audit logging, municipal scoping, theming, input validation, AI-service fallback and image checks passed. The objectives set in Chapter 1 were therefore met at the level of a working, tested prototype.

The main limitations are that SMS and WhatsApp delivery and offline synchronisation are simulated, that AI accuracy has been checked only on the test complaints, that results come from seeded data rather than live municipal operation, and that the system depends on a third-party AI service.

10.2 Learning Outcomes

- Understanding Generative AI concepts, including large language models, multimodal models and embeddings.
- Applying prompt engineering to obtain structured, validated output.
- Integrating an AI model or API into a backend service securely.
- Developing a complete multi-role web application with authentication, role-based access control, a database and a map interface.
- Evaluating AI-generated outputs and recognising the difference between confidence and accuracy.
- Understanding responsible AI practices such as human oversight, privacy and protection of API keys.
- Working collaboratively on a software project and documenting it.

BIBLIOGRAPHY / REFERENCES

The references follow one consistent style: author or organisation, title, publisher or source, year, and URL or identifier. Web sources that were opened during the project show the access date.

Policy documents, applications and research reports

[1] Ministry of Urban Development, Government of India. "Smart Cities Mission Statement & Guidelines." June 2015. https://www.ielrc.org/content/e1512.pdf (copy of the official document). Accessed 28 October 2026.

[2] Press Information Bureau, Government of India. "Centralised Public Grievance Redress and Monitoring System (CPGRAMS)." Backgrounder, 9 August 2026. https://static.pib.gov.in/WriteReadData/specificdocs/documents/2026/aug/doc202689948801.pdf. Accessed 28 October 2026. Portal: https://pgportal.gov.in/.

[3] Ministry of Housing and Urban Affairs, Government of India. "Swachhata-MoHUA" mobile application (official app of MoHUA). App Store listing: https://apps.apple.com/app/id1124033628. Accessed 28 October 2026.

[4] Peixoto, T. and Sifry, M. L. (eds.). Civic Tech in the Global South: Assessing Technology for the Public Good. World Bank and Civic Hall, 2017. http://hdl.handle.net/10986/27947.

[5] World Bank. "CivicTech: Transparency, Engagement, and Collaboration for Better Governance." Equitable Growth, Finance and Institutions Insight, World Bank, 2022. https://openknowledge.worldbank.org/bitstreams/b38b6480-237f-53f6-b670-7857a05b2403/download. Accessed 28 October 2026.

[6] World Bank. "Cross-Country Empirical Analysis of GovTech Platforms on Citizen Engagement." Policy Research Working Paper No. 10716, 2024. DOI: 10.1596/1813-9450-10716.

Books and papers

[7] Vaswani, A., et al. "Attention Is All You Need." Advances in Neural Information Processing Systems 30 (NeurIPS 2017), pp. 5998–6008, 2017.

[8] Goodfellow, I., Bengio, Y. and Courville, A. Deep Learning. MIT Press, Cambridge, Massachusetts, 2016.

[9] Sinnott, R. W. "Virtues of the Haversine." Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[10] Jones, M., Bradley, J. and Sakimura, N. "JSON Web Token (JWT)." RFC 7519, Internet Engineering Task Force, May 2015. https://www.rfc-editor.org/rfc/rfc7519.

Software, libraries and services

[11] MongoDB, Inc. MongoDB Manual, version 7.0. https://www.mongodb.com/docs/manual/.

[12] Mongoose. Mongoose ODM documentation, version 8. https://mongoosejs.com/docs/.

[13] OpenJS Foundation. Express.js documentation, version 4. https://expressjs.com/.

[14] Agafonkin, V. Leaflet: an open-source JavaScript library for interactive maps. https://leafletjs.com/.

[15] Meta Open Source. React documentation, version 18. https://react.dev/.

[16] Vite. Vite documentation, version 6. https://vite.dev/.

[17] Tailwind Labs. Tailwind CSS documentation, version 3.4. https://tailwindcss.com/docs.

[18] Recharts. Recharts documentation. https://recharts.org/.

[19] OpenStreetMap contributors. OpenStreetMap map data and tiles. https://www.openstreetmap.org/.

APPENDICES

Appendix A: Source Code | Appendix B: Prompt Library | Appendix C: Sample Input and Output | Appendix D: User Manual | Appendix E: Installation and Deployment Guide | Appendix F: Test Results | Appendix G: Dataset Description | Appendix H: AI Usage Disclosure

Appendix A: Source Code

A.1 Repository. https://github.com/PreranaShinde01/NagarSetu_AI. Key code excerpts are given in Section 7.4 (AI triage call, Haversine and cosine-similarity duplicate check, and the confidence rule). No credentials or private data are included.

A.2 API endpoint specification. Table A.1 lists the principal REST endpoints of the backend.

Table A.1: Key API Endpoints

Method	Endpoint	Description	Auth scope
POST	/api/v1/auth/login	Authenticate a user and return a JWT	Public
GET	/api/v1/auth/me	Return the authenticated user's profile and role	Authenticated
GET	/api/v1/municipalities	List active municipal corporations (IMC)	Public
GET	/api/v1/complaints	Filter and paginate complaints	Role scoped
POST	/api/v1/complaints	Submit a new citizen complaint	Citizen
GET	/api/v1/complaints/:id	Retrieve a complaint case file	Authenticated
POST	/api/v1/complaints/:id/route	Route a ticket to a department and prabhag	Controller
GET	/api/v1/work-orders	Fetch assigned work orders	Worker / supervisor
POST	/api/v1/work-orders	Dispatch a new work order to a field crew	Controller / supervisor
PUT	/api/v1/work-orders/:id/verify	Supervisor approval or rejection of the resolution	Department supervisor
GET	/api/v1/analytics/overview	Telemetry and resolution velocity	Staff / controller
GET	/api/v1/analytics/complaints	Inflow trends and category distribution	Staff / controller
GET	/api/v1/audit	Append-only governance audit log	Controller / admin

Appendix B: Prompt Library

Table B.1 lists the prompts used in the application, with their purpose, inputs and outputs. The verbatim triage prompt (P1) is given in Section 7.5.

Table B.1: Prompt Library

Prompt	Purpose	Inputs	Output
P1: Complaint triage	Intent classification, title, severity and confidence	Complaint text; allowed categories	Category, title, severity, confidence, summary
P2: Image assessment	Check civic relevance and identify damage pattern	Photograph	Validity flag and damage description
P3: Duplicate embedding input	Text prepared for semantic similarity	Complaint description	Embedding vector
P4: Civic assistant (system prompt)	Guide filing, status lookups and civic questions in English and Marathi	Chat messages; complaint ID	Conversational reply

Table B.2: Prompt Refinement Log

Version	Change made	Reason	Effect on output
v1	Initial prompt for basic civic complaint classification and response generation.	The initial design did not explicitly require confidence scores, severity levels, or structured output.	Basic complaint categorization and descriptive responses, with potentially inconsistent formatting.
v2	Refined prompts with defined input parameters, JSON output, confidence scores, severity levels, category restrictions, and human-review instructions.	To improve consistency, reduce unsupported AI conclusions, and support integration with NagarSetu AI's complaint management workflow.	More structured, explainable, and application-ready responses, including priority recommendations and confidence information.

Appendix C: Sample Input and Output

Table C.1: Sample Input and Output (Test Case TC-03)

Item	Content
User input	Citizen complaint: "Pothole on Station Road, Prabhag 4", with a photograph.
Observed output	Complaint created with an ID such as NGS-2026-000143; AI analysis returned about 94% confidence and priority P2.
Remaining fields	A citizen reported a pothole on Station Road in Prabhag 4, potentially affecting road safety and vehicle movement. The complaint was categorized as a road infrastructure issue and assigned a recommended P2 priority for municipal review.
Edited or post-processed?	The suggested title, category and summary are shown as examples phrased for clarity and consistency. The confidence and priority values are those returned in test case TC-03.

Table C.2: Sample Civic Assistant Exchange

Turn	Content
Citizen	Hello, I want to check the status of my complaint NGS-2026-0042 regarding a pothole on Station Road, Prabhag 4.
Assistant	Hello! Welcome to NagarSetu AI Civic Assistant. I can help you track your complaint NGS-2026-0042. Please sign in to your account to verify access. Once verified, I can retrieve the current complaint status, assigned department, and latest updates.

Appendix D: User Manual

D.1 Citizen

1. Sign in to the Citizen Portal.
2. Choose "File complaint" (or open the NagarSetu Assistant) and describe the problem in text or voice, attach a photograph and confirm the location.
3. Review the suggested title and category and submit. Note the complaint ID.
4. Track the status timeline. When work is complete, confirm or reject the resolution. A rejection reopens the complaint.

D.2 Field worker

1. Open the assigned task list and accept a work order.
2. Use the navigation link to reach the site and choose "Start work".
3. After completion, upload the before-and-after photographs with notes. The GPS position must be within 50 m of the reported location.

D.3 Ward and department officers

1. Open the ward or departmental queue and view the complaints in scope.
2. Department officers review the proof of work and approve it or reject it for rework.

D.4 Controller, senior authority and administrator

- The controller reviews the AI triage queue, approves or overrides the recommended route, dispatches work orders, monitors SLA alerts and handles escalations, and can open the audit log.

- The senior authority views KPI and city health dashboards and configures SLA policy.
- The system administrator manages municipalities, users and the RBAC configuration.

Appendix E: Installation and Deployment Guide

E.1 Prerequisites

- Node.js v20 or higher with npm v10 or higher.
- MongoDB Community Server v7.0 or higher (or a MongoDB Atlas cluster).
- A supported web browser (Chrome, Edge, Firefox or Safari).
- An API key for the Generative AI provider .

E.2 Configuration

Create an environment file for the backend (excluded from version control) with the settings in Table E.1. Real values must never be shared or committed.

Table E.1: Environment Settings

Setting	Purpose	Variable name in the project
Database address	MongoDB connection (local: `mongodb://127.0.0.1:27017/nagarsetu_ai`)	MONGODB_URI
JWT secret	Signs authentication tokens	JWT_SECRET
AI API key	Access to the Generative AI service	AI_API_KEY
Server port	Port of the Express.js server	PORT

E.3 Commands

```
# 1. Start MongoDB (local installation)
mongod --dbpath <data-directory>

# 2. Backend
cd server
npm install
node src/seed/seed.js        # load the Ichalkaranji hierarchy and demonstration data
npm start                    # or the start script defined in package.json

# 3. Frontend (from the project root, in a second terminal)
npm install
npm run dev                  # Vite development server; use the build script for
production
```

Appendix F: Test Results

The detailed test cases and their outcomes are listed in Table 8.1. The endpoints exercised during API integration testing were `/auth`, `/complaints`, `/work-orders`, `/analytics` and `/audit`.

Appendix G: Dataset Description

Table G.1: Seeded Dataset

Item	Description
Source	Created by the project team with the seed script (`server/src/seed/seed.js`) for the Ichalkaranji pilot configuration.
Format	MongoDB documents in the collections of Section 5.5.
Size	12 prabhags, 4 departments, 11 demonstration users, more than 44 complaints, 29 work orders and 74 audit records at seeding.
Preprocessing	Complaints and work orders are linked across prabhags with consistent identifiers, statuses and SLA values.
Permissions and privacy	Demonstration users are fictitious personas; no real citizen personal data is stored in the dataset.

Appendix H: AI Usage Disclosure

Generative AI tools, including ChatGPT and Google Antigravity, were used during the development of **NagarSetu AI** for project planning, code generation and assistance, debugging, UI/UX design, database and API development guidance, prompt engineering, and technical documentation.

This report was prepared with the assistance of Generative AI using project requirements, development notes, and implementation details. AI-generated suggestions and outputs were reviewed and refined during development. The final implementation, testing results, and documentation were verified by the project team. External sources have been acknowledged wherever applicable.

Github Repository QR Code

https://github.com/PreranaShinde01/NagarSetu_AI